

20260807-大模型并行策略解析

目录

- 一、概述
- 二、并行策略分类体系
- 三、六大并行策略解析
 - 3.1 DP — Data Parallelism (数据并行): 从单卡计算拓展到多副本
 - 3.2 TP — Tensor Parallelism (张量并行): 沿张量维度拆分单层计算
 - 3.3 PP — Pipeline Parallelism (流水线并行): 沿网络深度拆分模型
 - 3.4 SP — Sequence Parallelism (序列并行): 消除 TP 组内的激活冗余
 - 3.5 CP — Context Parallelism (上下文并行): 切分 Attention 的上下文
 - 3.6 EP — Expert Parallelism (专家并行): 沿专家维度扩展 MoE 模型
- 四、SGLang推理场景的混合并行
 - 4.1 SGLang各种并行的rank划分
 - 4.2 SGLang DSV4 使用 CP并行的条件
 - 4.3 CP的切分方式zigzag / interleave
 - 4.4 DSV4的CP+EP混合并行
 - 4.4.1 整体结构
 - 4.4.2 CP+EP完整路径 (以--tp4 --cp 4 --ep 4为例)
 - 步骤一: token数padding到cp_size的倍数
 - 步骤二: CP 准入判断
 - 步骤三: 建 attn_cp_metadata
 - 步骤四: attention metadata按照cp_rank切片
 - 步骤五: Tensor按 cp_rank::cp_size 切分
 - 步骤六: 每个 rank 只算自己那 2 个 token 的 KV
 - 步骤七: KV 跨 CP 组 AllGather (核心通信 ①)
 - 步骤八: 写完整 KV cache, 然后算 attention
 - 步骤九: 进 MoE 前 hidden AllGather (核心通信 ②, 仅 a2a=none)
 - 步骤十: MoE forward
 - forward_normal()
 - forward_deepep()
 - 步骤十一: 出 MoE 后 ReduceScatter 回收 (核心通信 ③, 仅 a2a=none)
 - 步骤十二: 末层输出 AllGather 还原 (核心通信 ④)
- 附录
 - A. Ring-AllReduce通信

- B. SGLang 各种并行size和rank计算
- C. DSV4 TP+DP+PP+EP混合并行
 - 1 整体结构
 - 2 MQALayer: Attention 的 6 个权重
 - 1. 前置: 四种并行线性层
 - 2. forward 路径
 - 3. 每个权重的定义代码对照
 - 4. 为什么 wo_b 的 AllReduce 是有条件的? 为什么 DP-Attention 要显式 all_reduce?
 - 3 DeepseekV2MoE (DSV4复用之前的结构): MoE 的两条路
 - 路径 A: forward_normal (纯 TP / TP+DP, 无 EP)
 - 路径 B: forward_deepep (启用 EP + DeepEP AllToAll)
 - 4 embed_tokens 与 lm_head
- D. 通信原语汇总
 - 1对多的基础原语
 - 多对多的组合原语
- 参考资料

一、概述

大语言模型 (LLM) 规模从百亿级向千亿级乃至万亿级突破, 单张 GPU 已无法容纳完整模型。分布式并行策略成为支撑大模型训练和推理的核心基础设施。

随着模型参数、上下文长度和请求规模增长, 单卡会依次遇到四类瓶颈:

1. 算得不够快: 单卡吞吐有限, 需要让多卡处理不同数据;
2. 模型放不下: 单层权重或完整模型超过单卡显存;
3. 序列放不下: 激活、KV Cache 和 Attention 计算随序列增长;
4. 稀疏模型不好放: MoE 专家数量巨大, 且路由负载不均。

二、并行策略分类体系

并行策略总体上可分为两大类:

1. 数据并行 (DP) — 将输入数据切分到不同设备, 每台设备持有完整模型副本
2. 模型并行 (MP) — 将模型参数 / 计算切分到不同设备, 细分为:
 - 按权重维度切分: TP (张量并行)、EP (专家并行)
 - 按序列维度切分: SP (序列并行)、CP (上下文并行)
 - 按层维度切分: PP (流水线并行)

并行策略可以按切分维度理解:

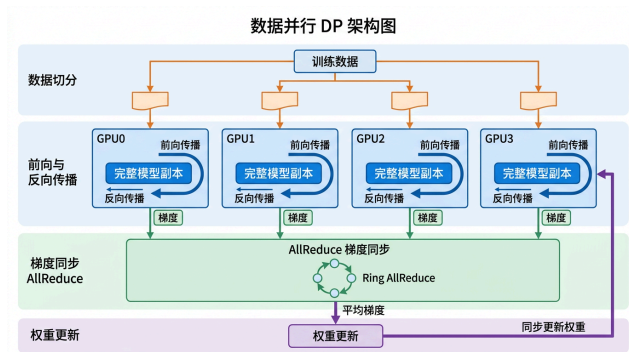
1	策略	切分对象	典型通信	解决的首
2	DP	Batch / Request	训练时 AllReduce；推理副本间通常不通信	提高吞吐
3	TP	Hidden / Head / Weight	AllReduce、AllGather、ReduceScatter	单层权重
4	PP	Transformer Layer	P2P Send/Recv	完整模型
5	SP	Sequence 上的激活	AllGather、ReduceScatter	TP rank 的
6	CP	Attention Context	Ring P2P、AllGather、ReduceScatter	超长上下
7	EP	MoE Expert	AllToAll	专家参数

因此不存在一种“最强并行策略”，每种策略都对应一个主要矛盾，它们分别作用于数据、权重、层、序列、上下文和专家等不同维度，并在真实系统中共同构成混合并行。

通信方式的具体操作可以参考附录D

三、六大并行策略解析

3.1 DP — Data Parallelism（数据并行）：从单卡计算拓展到多副本



当模型可以再单卡上放置，但单卡Batch和吞吐有限时，最直接的方案就是复制模型，让不同的设备处理不同的数据。

- i 实现DP时，每张卡保存完整模型：**
- 输入数据或推理请求按 DP rank 切分
 - 训练场景各卡独立计算局部梯度，再通过Ring-AllReduce 保持参数一致（Ring-AllReduce可以参考附录A）
 - 推理场景通常各自处理请求，DP 组内无需通信

适用场景： 模型能放入单卡内存时 / 高并发吞吐场景 / 离线批量训练

优缺点：

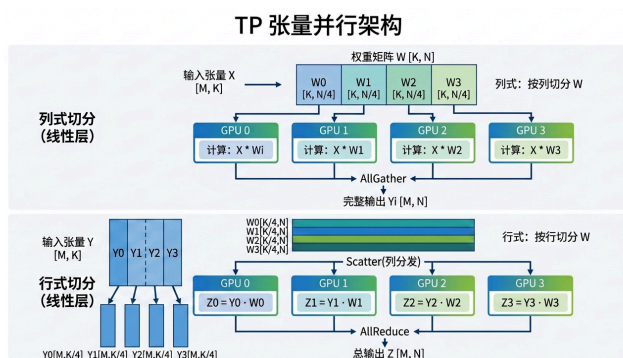
- 优点：实现简单，吞吐量线性扩展，通信开销低

- 缺点：每卡必须装完整模型，显存要求最高

主要框架实现：

- vLLM：多实例部署，K8s/Gateway 负载均衡分发请求
- SGLang：类似的多副本部署
- DeepSpeed：ZeRO Stage 1-3 优化 DP 显存效率
- TensorRT-LLM：--data-parallel-size 配置

3.2 TP — Tensor Parallelism (张量并行)：沿张量维度拆分单层计算



当模型大到单卡无法保存，DP的完整副本成为瓶颈。

Transformer中权重最重的部分是Linear和Attention，这两个部分可以分别沿Hidden和Head维度拆分到多张GPU上并行计算，每张卡只存部分权重。

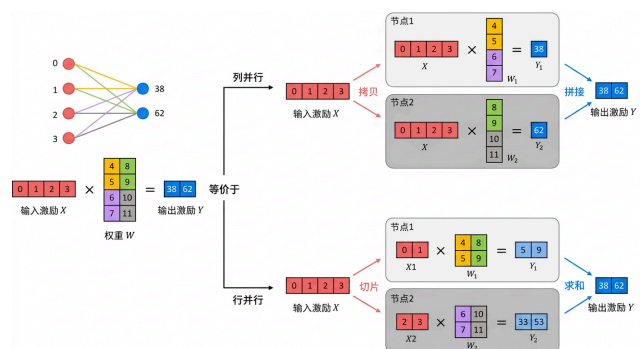
切分原理 (矩阵乘法 $Y = XW$)

- 按列切分 (Column-wise)： $W = [W_1 | W_2 | \dots | W_n]$ ，每个 GPU 计算 XW_i ，AllGather 拼接完整输出
- 按行切分 (Row-wise)： $W = [W_1; W_2; \dots; W_n]$ ，ReduceScatter 分发输入分片，各卡计算后 AllReduce 汇总

MLP层：

- 第一层Linear (列切)：各卡拿着完整的输入 X 跟列切片计算，不需要通信
- 第二层Linear (行切)：列切的计算结果，用来跟行切片计算出局部结果，最后进行AllReduce将局部结果求和

Attention层：多头按head维度切分到每一张卡上（列切），各卡负责部分head，独立计算QKV和Attention



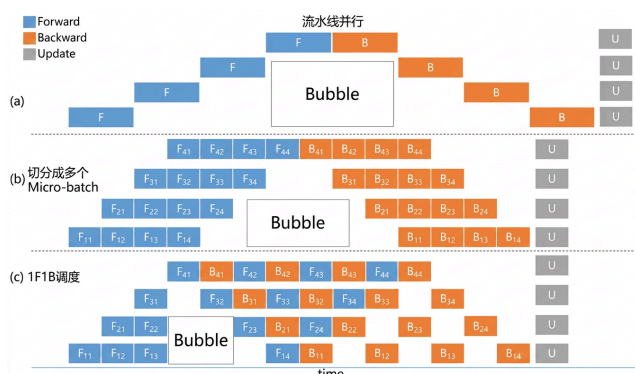
适用场景：低延迟在线推理 / 节点内 NVLink 互联部署

优缺点：

- 优点：降低了每张卡的权重和计算量
- 缺点：每个Transformer层都会通信，对 NVLink 互联的方式依赖大，跨节点TP会被带宽和延迟限制

主要框架实现：vLLM (--tensor-parallel-size N)、TensorRT-LLM（支持 Attention DP + TP 混合）、SGLang (--tp N)

3.3 PP — Pipeline Parallelism（流水线并行）：沿网络深度拆分模型



TP的高频通信适合节点内高速互联，却不适合无限跨节点拓展。

PP将模型按层切分到不同GPU，数据像流水线在各设备间一次流转。

❗ 朴素层间穿行会导致大量GPU等待，产生很多的空泡（Bubble），降低GPU资源利用率

优化思路：

1. 使用mini-batch进行计算，提高计算密集度，减少空泡
2. 1F1B调度：训练场景下，前向一个mini-batch后立即进行反向计

算，降低中间激活显存

适用场景： 超大模型训练/推理 / 跨界点部署 / 机器持有非2次幂的GPU数量

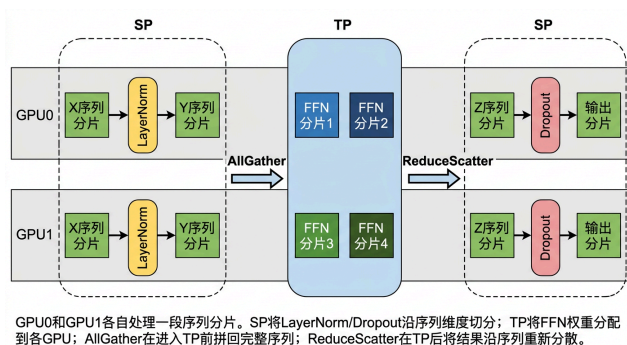
优缺点：

- 优点：支持超大模型的跨节点训练/推理
- 缺点：存在流水线空泡，stage负载不均和调度复杂的问题

主要框架实现： vLLM (--pipeline-parallel-size N)、TensorRT-LLM

(pipeline_parallel_size)、Megatron-LM、SGLang (Chunked Pipeline)

3.4 SP — Sequence Parallelism (序列并行)：消除 TP 组内的激活冗余



TP 切分了 Linear 权重，但 LayerNorm、Dropout、残差连接等操作的激活，仍可能在每个 TP rank 上完整保存。长序列下，这些重复激活会成为显存瓶颈，迫使系统进行代价较高的全量激活重算。

i SP 将序列维度上彼此独立的激活层分片计算，随后发送给不同的 TP rank：

- 进入需要完整输入的 TP 区域前，用 AllGather 拼接；
- 离开 TP 区域时，用 ReduceScatter 汇总并重新切分；
- LayerNorm、Dropout、残差等操作只处理本地序列分片。

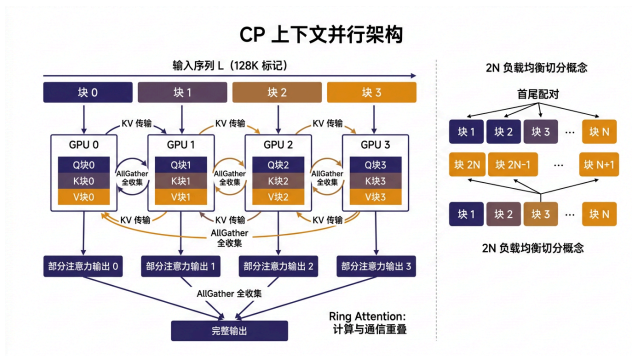
适用场景： 应用TP且具备长上下文

优缺点：

- 优点：和TP绑定使用，重新组织已有通信，减少激活冗余
- 缺点：没有切开 Attention 内部的全上下文依赖，因此不能独立解决超长上下文问题

主要框架实现：vLLM (torch.compile 自动识别替换)、Megatron-LM (原生 TP-SP)

3.5 CP — Context Parallelism (上下文并行)：切分 Attention 的上下文



当上下文从 32K 增长到 128K、1M 甚至更长时，问题不再只是外围激活：KV Cache 随序列长度线性增长，传统 Attention 的计算和中间状态增长更快。SP 只切 LayerNorm 等局部操作，无法解决 Attention 需要访问全局上下文的问题。

CP 的基本计算过程如下：

- 把输入分成cp_size个块，每块输入到不同的cp_rank中处理
- 通过 AllGather 让本地 Q 依次看到全局 KV；
- 对因果 Attention 进行负载均衡，避免后半段 rank 计算更重。
 - [1, 2N]一组，使用 $0 + 2N - 1 = 2N - 1$ 的KV Cache
 - [2, 2N-1]一组，使用 $1 + 2N - 2 = 2N - 1$ 的KV Cache
 - 确保每一组的计算量接近

与 SP 的关键区别

1	对比项	SP
2	定位	TP 的配套优化
3	切分范围	非 TP 算子的序歹
4	是否处理全局 Attention	否

适用场景：超长上下文推理（128K+ tokens） / 长篇合同、论文、代码库分析

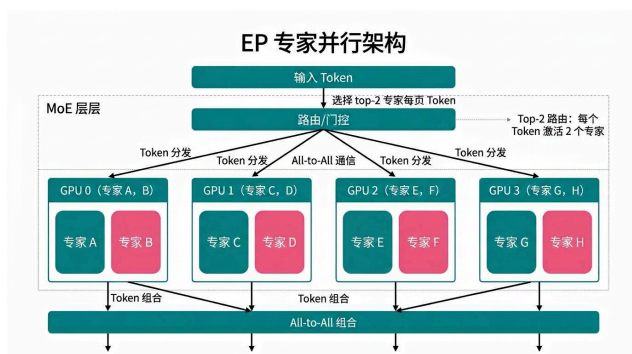
- 优缺点：
- 优点：每张卡只计算自己rank上的序列，有效解决长上下文推理带来的问题

- 缺点：需要改造Attention内核，不同模型的KV结构也会影响切分方式，获取全量KV Cache需要AllGather通信

主要框架实现：SGLang（CP 已合入主分支）、TensorRT-LLM

(context_parallel_size)、vLLM (PCP, DCP)

3.6 EP — Expert Parallelism (专家并行)：沿专家维度扩展 MoE 模型



稠密模型增加参数通常也会增加每个 token 的计算量。MoE 通过“拥有大量专家、每个 token 只激活 top-k 个专家”解耦总参数数量和单 token 计算量。但如果所有 GPU 都保存所有专家，专家参数会再次成为显存瓶颈。

核心思想： MoE 模型中将不同专家分布到不同 GPU，每卡只存部分完整专家权重。

- **TP-only：** 每个专家权重切片到所有卡，AllReduce 通信，每个 GPU 要持有所有专家的切片。N 个专家 × TP 切片数 P，每卡仍需存储大量参数——这是为什么纯 TP 在大规模 MoE 上不实用的根本原因
- **EP-only：** 完整专家在单卡，每个 Token 按 router 输出选 top-k 个专家后，通过 All-to-All 发往目标专家所在卡；返回时再发回原卡聚合结果
- **ETP 混合：** 先按 EP 分组把不同专家放到不同的设备组上，组内再用 TP 把单个专家的权重矩阵沿隐藏维度切分
- **专家负载均衡：** 热门专家 Token 数可能是冷门专家数十倍。
 - 离线 EPLB、在线 EPLB、Auxiliary Loss、Wide-EP

适用场景：大规模的MOE模型训练/推理

优缺点：

- 优点：EP 将不同专家放到不同设备，参数不再重复
- 缺点：token 必须按路由结果跨卡发送，计算后也需要从不同专家发回原路径

主要框架实现：

- vLLM：--enable-expert-parallel，通过 flatten_tp_across_dp 映射专家
- TensorRT-LLM：TP-only / EP-only / Hybrid ETP / Wide-EP 四种模式
- SGLang：原生 EP 支持
- DeepSeek：Deep_EP 自定义 EP 通信 Kernel

四、SGLang推理场景的混合并行

4.1 SGLang各种并行的rank划分

</> sglang各种并行rank划分

Bash

```
1 # MOE和Attn走的是不同的划分方式，
2 # 基本上--dp == attn_dp
3 # attn_tp 和 moe_tp需要在其他并行模式划分完成后才能确定size，最少为1
4 总的tp_rank (--tp)
5 |— Attention 层级
6 |   |— dp (attn_dp_size)
7 |       |— cp (attn_cp_size)
8 |           |— attn_tp (attn_tp_size)
9 |— MoE 层级
10     |— moe_dp (moe_dp_size)
11         |— ep (ep_size)
12             |— moe_tp (moe_tp_size)
```

具体的计算代码详见附录B

4.2 SGLang DSV4 使用 CP并行的条件

server_args.py 的 _handle_context_parallelism 方法对CP得使用进行了检查

- 启动参数:

- 非DSA的模型, `--enable-prefill-cp` 和 `--cp-strategy` (`zigzag` / `interleave`)
- DSA模型需要开启 `--enable-prefill-context-parallel`

- 并行Size:

- `tp_size % attn_cp_size == 0`: `world_size`需要被CP的size整除
- `tp_size % (dp_size * attn_cp_size) == 0`: DP x CP不能超过word_size

`deepseek_v4_hook.py` 的 `validate_deepseek_v4_cp` 函数限制了DSV4的CP使用条件

- 启动参数:

- 使用 `--enable-prefill-context-parallel`, 并且设置 `--dsa-prefill-cp-mode="round-robin-split"`

- 并行Size:

- `dp_size==1`, 不支持在 `round-robin-split` 模式下使用dp attention
- `tp_size<=8`, CP只支持单机执行, 跨机CP有精度问题

`server_args.py` 的 `_handle_model_specific_adjustments` 方法对CP并行参数进行了修改

</> server_args.py

Python

```
1 if self.enable_dsa_prefill_context_parallel:
2     if self.dsa_prefill_cp_mode == "in-seq-split":
3         self.enable_dp_attention = True
4         self.moe_dense_tp_size = 1
5         self.moe_a2a_backend = "deepep"
6         self.ep_size = self.tp_size
7     else: # round-robin-split
8         self.moe_dense_tp_size = 1
9         assert (
10             self.dp_size == 1
11         ), "For round-robin split mode, dp attention is not supported."
12         self.attn_cp_size = self.tp_size // self.dp_size
13         self.disable_pieewise_cuda_graph = True # PCG禁用
```

4.3 CP的切分方式zigzag / interleave

在SGLang中, `--cp-strategy` 和 `--dsa-prefill-cp-mode` 可以用于控制CP的切分策略, 当前支持两种策略: `round-robin-split` 和 `in-seq-split`

- **round-robin-split (interleave, contiguous)**: `token_idx % cp_size`, rank i拿到所有下标模 `cp_size` 等于 i 的 token, 以 `cp_size=4` 为例


```

1 原始顺序: token0 token1 token2 token3 token4 token5 token6 token7
  token8 token9 token10 token11 ...
2
3 rank0: token0, token4, token8, ...
4 rank1: token1, token5, token9, ...
5 rank2: token2, token6, token10, ...
6 rank3: token3, token7, token11, ...

```

NSA / DSA 的长 prefill 路径不是普通 dense attention。它有 indexer、topk、ragged metadata、`seqLens_expanded`、`page_table` 等额外结构。这里的第一性约束变成：

- ❗ 不仅 query hidden states 要切，和 query token 一一对应的 NSA indexer 元数据也必须同步切。

round-robin 的规则是 $\text{token_idx} \% \text{cp_size} == \text{cp_rank}$ 。它牺牲了连续片段的 locality，但换来两个特性：

- 对长序列天然负载均衡，每个 rank 均匀抽样不同位置的 token。
- 可以用 `dsa_cp_round_robin_split_q_seqs()` 处理多 request 情况：某个 rank 对某些短 request 可能没有 token，于是需要返回 `bs_offset` 和 `bs_idx` 过滤 batch 维元数据。

</> 举个例子

```

1 seq0 = ABCDEFG
2 seq1 = 123
3 CP 拼接request ----> seq = ABCDEFG123
4 round-robin切分后
5 # 通过bs_idx确认当前token所属的batch, bs_offset确定batch从第几个token开始
6 rank0: AE2 ----> bs_idx[0,1] bs_offset[0, 2]
7 rank1: BF3 ----> bs_idx[0,1] bs_offset[0, 2]
8 rank2: CG ----> bs_idx[0] bs_offset[0]
9 rank3: D1 ----> bs_idx[0,1] bs_offset[0,1]

```

• in-seq-split (zigzag, two_chunks):

以 `cp_size=4` 为例，一个请求会被切成 $2 \times 4 = 8$ 段：

```

1 原始顺序: block0 block1 block2 block3 block4 block5 block6 block7
2 zigzag:  block0 block7 block1 block6 block2 block5 block3 block4
3
4 rank0: block0 + block7
5 rank1: block1 + block6

```



```
6 rank2: block2 + block5
7 rank3: block3 + block4
```

这样做是为了平衡 causal attention 的计算量。靠前 query 能看到的历史 KV 少，靠后 query 能看到的历史 KV 多；把一个靠前段和一个靠后段配到同一个 rank，可以让各 rank 的 attention 计算更均衡。

4.4 DSV4的CP+EP混合并行

4.4.1 整体结构

组网结构

</> Plain Text |

```
1 DeepseekV4ForCausalLM
2 |— embed_tokens:      VocabParallelEmbedding      ← 词表切分
3 |— layers[0..42]:     DeepseekV4DecoderLayer x43
4 |   |— mHC pre + input_layernorm                    ← 替代普通残差，本节略
5 |   |— self_attn:      MQALayer                    ← Attention (本章重点)
6 |   |— mHC post + FFN 前 pre
7 |   |— mlp:            DeepseekV2MoE                ← MoE (本章重点)
8 |   |— HC 收尾 (延迟到下一层融合)
9 |— hc_head + norm
10 |— lm_head:          ParallelLMHead                ← 词表切分 + AllGather
```

Attention组网

</> Attention组网 Plain Text |

```
1 组网逻辑 python/sglang/srt/models/deepseek_v4.py MQALayer.forward
2  _____
3 hidden (full, replicated)
4 |
5 |— wqkv_a (ReplicatedLinear)      ← 不切，每人都算完整低秩投影
6 |   q_lora = hidden @ wqkv_a[:q_lora_rank]      → [tokens, 1024]
7 |   kv      = hidden @ wqkv_a[q_lora_rank:]      → [tokens, 512]
8 |
9 |— q_norm      ← 不切
10 |
11 |— wq_b (ColumnParallelLinear)    ← 列切: n_heads × head_dim 沿 head 维切
12 |   q = q_a @ wq_b                → 每个 attn_tp rank 持有
   n_heads/attn_tp 个 head
13 |
```

```

14  |— fused_qk_norm_rope_swa_store → 计算Q
15  |— _compute_kv                    → 计算KV
16  |
17  |— [Attention kernel]              ← 不发起集合通信! 各 rank 算自己的 head
18  |   flash_mla_with_kvcache(...) → o = [tokens, local_heads ×
    head_dim]
19  |
20  |— 反 RoPE: fused_rope_inplace(inverse=True)
21  |
22  |— wo_a (ColumnParallelLinear)    ← 列切: n_groups × o_lora_rank 沿
    group 维切
23  |   o_compressed = o @ wo_a      → [tokens, local_groups ×
    o_lora_rank]
24  |
25  |— wo_b (RowParallelLinear)      ← 行切: 输入维切, ★ AllReduce ★
26  |   output = o_compressed @ wo_b
27  |   if attn_tp_size < global_tp_size:
28  |       output = attn_tp_all_reduce(output) ← DP-Attention 时显式
    reduce

```

MOE组网 (DeepEP)

</> MOE

Plain Text

```

1  组网逻辑
2  hidden (replicated)
3  |
4  |— shared_expert (★ TP1, 复制不切 ★) → shared_output (完整)
5  |   _shared_expert_use_tp1=True
6  |
7  |— gate (replicated) → router_logits
8  |— topk → 选 6 个专家 (全局 expert id)
9  |
10 |— ★ dispatch AllToAll ★ ← 把 token 发给"持有目标 expert 的 rank"
11 |
12 |— experts 本地计算 (每个 rank 只有 256/ep 个完整 expert)
13 |   w13/w2 不切中间维, expert 整体持有
14 |
15 |— ★ combine AllToAll ★ ← 把结果发回源 rank 并累加
16 |
17 |— final = combined + shared_output ← 无 AllReduce!

```

4.4.2 CP+EP完整路径 (以--tp4 --cp 4 --ep 4为例)

假设原始请求是一条 8 token 的序列: `[t0, t1, t2, t3, t4, t5, t6, t7]`。CP size=4, 4 张卡各拿 2 个 token

步骤一: token数padding到cp_size的倍数

CP 的所有切片都假设「总 token 数能被 `cp_size` 整除」, 这个前提由 `ForwardBatch.prepare_mlp_sync_batch` 保证, 发生在 attention 元数据构建和 `model.forward` 之前。

</> forward_batch_info.py:1099-1104

Python

```
1 # prepare_mlp_sync_batch()
2 cp_align_size = get_cp_padding_align_size()      # round-
  robin=cp_size(4); zigzag=2*cp_size(8)
3 for i in range(sync_group_size):
4     # [CP] 这一行定死本步的 padded token 数
5     global_num_tokens[i] = ceil_align(global_num_tokens[i],
  cp_align_size)
```

补数据在 `_pad_inputs_to_size` (`forward_batch_info.py`):

- `input_ids` / `out_cache_loc` / `positions` 按 token 维补 0,
- `seq_lens` 按 batch 维补 cuda-graph fill value。

i `extend_seq_lens` 只补 batch 维。于是后面同时存在两个“token 数”:

- `len(input_ids) == out_cache_loc.shape[0]` → **padded 长度** (4 的倍数)
- `sum(extend_seq_lens_cpu)` → **真实 token 数**

步骤二: CP 准入判断

不是每一批都能走 CP。 `can_dsa_cp_split` (`dsa/utils.py`) 六个条件全满足才开, 否则整批回退非 CP:

</> dsa/utils.py

Python

```
1 # can_dsa_cp_split
2 if is_dsa_prefill_cp_round_robin_split():
3     cur_cp_seq_len = seq_len // cp_size
4     assert seq_len % cp_size == 0          # 依赖步骤1 的 padding
5 return (
6     cur_cp_seq_len != 0
7     and cp_size > 1
8     and use_dsa
```

```

9     and forward_batch.forward_mode.is_context_parallel_extend() #
    EXTEND / MIXED
10     and is_dsa_enable_prefill_cp()
11     and sum(forward_batch.extend_seq_lens_cpu) >= cp_size      # 太短
    不值得切
12 )

```

步骤三：建 `attn_cp_metadata`

```

</> python/sglang/srt/models/deepseek_v4.py Python |

1 # DeepseekV4ForCausalLM.forward
2 if self.dsa_enable_prefill_cp:
3     if can_dsa_cp_split(len(input_ids), self.cp_size, True,
        forward_batch):
4         forward_batch.attn_cp_metadata =
        prepare_context_parallel_metadata(
5             len(input_ids),          # ← padded 长度
6             self.cp_rank, self.cp_size,
7             forward_batch.seq_lens_cpu.tolist(),
8             extend_seqs_len=forward_batch.extend_seq_lens_cpu,
9         )

```

round-robin 模式下这个函数在 `cp_utils.py` 直接返回一个空的 `ContextParallelMetadata()` ——它只是「CP 已开启」的标记位，下游 `dsa_use_prefill_cp` 靠判空来决定走不走 CP 分支；真正的切分靠索引运算，不需要置换表。只有 in-seq-split (zigzag) 才会生成整套 `split_list` / `zigzag_index` / `cp_reverse_index`。

步骤四：attention metadata按照cp_rank切片

构建好 `attn_cp_metadata` 后，会进行 metadata 的切片

```

</> python/sglang/srt/models/deepseek_v4.py Python |

1 # DeepseekV4ForCausalLM.forward
2 if self.dsa_enable_prefill_cp:
3     if can_dsa_cp_split(len(input_ids), self.cp_size, True,
        forward_batch):
4         forward_batch.attn_cp_metadata =
        prepare_context_parallel_metadata(...)
5         if is_dsa_prefill_cp_round_robin_split():
6             attn_backend = get_attn_backend()
7             metadata = attn_backend.forward_metadata

```

```
8         core_meta = metadata.core_attn_metadata
9         core_meta.apply_cp_reindex() # 切分元
数据
10        core_meta.init_flashmla_related(is_prefill=True)
11        #   ↳ c4_sparse_topk_lengths = clamp(c4_topk_lengths_clamp1,
max=512/1024)
12        #   ↳ c4_sparse_page_indices 全填 -1 占位, 等 indexer 写入
13        #   ↳ _pad_last_dim 把最后一维补到 64 倍数 (填 -1)
```

DSV4 的 attention metadata 有一堆 per-token 字段

(seq_lens_casual 、 positions_casual 、 page_table 、 swa_page_indices 、 c4_topk_lengths_* ...)。这些元数据是在 model.forward 之前按全局 token 建好的 (deepseek_v4_backend.py:1098)，所以要在模型入口前切片。

</> python/sglang/srt/layers/attention/deepseek_v4_backend.py

Python

|

```
1 # DeepseekV4ForCausalLM.forward 调用
2 def apply_cp_reindex(self) -> None:
3     cp_rank = get_parallel().attn_cp_rank
4     cp_size = get_parallel().attn_cp_size
5     idx = slice(cp_rank, None, cp_size) # ↳ 关键：步长切片
6     pre_global_len = self.seq_lens_casual.shape[0]
7     assert pre_global_len % cp_size == 0 # 全局 token 数
必须被 cp_size 整除
8     for field_name in self._CP_REINDEX_FIELDS: # 9 个字段逐个切
9         val = getattr(self, field_name, None)
10        setattr(self, field_name, val[idx].contiguous()) # 本地化
11    # _CP_GLOBAL_FIELDS 不切, 保持全局 (给 compressor 写路径用)
```

变量的切片方式：

1	类别	字段	原因
2	裁成 1/4	seq_lens_casual positions_casual page_table swa_page_indices swa_topk_lengths c4_topk_lengths_raw c4_topk_lengths_clamp1 c128_page_indices c128_topk_lengths_clamp1	都是 per-query-token 的： query

3	保持全局	raw_out_loc swa_out_cache_loc c4_out_loc c128_out_loc	都是 KV 写入位置；KV c 仍要覆盖完整 8 个 token
---	------	---	---

步骤五：Tensor按 `cp_rank::cp_size` 切分

DeepseekV4ForCausalLM.forward完成attention_metadata的切分后，会进入到DeepseekV4Model.forward中，其会对输入进行切分，每个rank把全局Tensor按`cp_rank::cp_size`切片，让自己拿到属于自己的token。

```
</> deepseek_v4.py Python |
1 # DeepseekV4Model.forward
2 if dsa_use_prefill_cp(forward_batch):
3     # [CP] 张量层面的 token 切分，规则同 apply_cp_reindex
4     if self.pp_group.is_first_rank:
5         hidden_states = cp_split_and_rebuild_data(forward_batch,
6             hidden_states)
7         positions = cp_split_and_rebuild_position(forward_batch, positions)
8         input_ids = cp_round_robin_input_ids(input_ids)
9         input_ids_global = input_ids
```

代码中切分了hidden_states, position, input_ids三个Tensor，本质上是对不同Tensor做相同的操作，切分示意（8 token、cp_size=4）：

```
</> Plain Text |
1 原始: [t0, t1, t2, t3, t4, t5, t6, t7]
2     _____
3 CP round-robin 切分:
4 GPU0(cp_rank=0): [t0, t4]    ← 拿第 0,4 个
5 GPU1(cp_rank=1): [t1, t5]    ← 拿第 1,5 个
6 GPU2(cp_rank=2): [t2, t6]
7 GPU3(cp_rank=3): [t3, t7]
```

`input_ids` 有两套走法：

```
</> cp_utils.py Python |
1 def cp_round_robin_input_ids(input_ids):
2     cp_size = get_attention_cp_size()
3     cp_rank = get_attention_cp_rank()
4     if get_moe_a2a_backend().is_none():
```



```

5         input_ids = input_ids.reshape(-1, cp_size).T.flatten() # 无
    A2A: 只换序, 仍是全量
6     else:
7         input_ids = input_ids[cp_rank::cp_size].contiguous() # 有
    A2A: 真的只留 1/4
8     return input_ids

```

- 无 A2A (FusedMoE): MoE 阶段每个 rank 会 gather 回全量 token, 所以 `input_ids` 也保持全量, 只是重排成「各 rank 段连续」的顺序。
- 有 DeepEP: token 由 dispatch a2a 按 expert 重新分布, 本 rank 只需要自己那一份。

步骤六: 每个 rank 只算自己那 2 个 token 的 KV

输入Tensor切分完成后, 进入MQALayer.forward, 在这个function中, wqkv_a 复制 (不切), 所以每个 rank 都能算出自己本地 token 的 q_a、kv_a 低秩投影。 `_compute_kv_bf16` 算出 bf16 的 KV (CP 路径故意保持 bf16 而不是直接写 FlashMLA cache, 因为后面要 all-gather)。

```

</> python/sglang/srt/models/deepseek_v4.py Python |
1 # Attention的部分了
2 # MQALayer._forward_prepare
3 if use_cp:
4     # [CP] 非 CP 路径 _compute_kv_to_cache 把 rmsnorm+RoPE+写 paged cache
5     # 融进一个 kernel; 但本 rank 只有 1/4 token 的 KV, 直接写会让 cache 残缺。
6     kv = self._compute_kv_bf16(x_linear, positions, qkv_a=qkv_a) # 只
    算 2 个 token
7     kv = cp_all_gather_rerange_output( # ←
    通信 ①
8         kv.contiguous(), self.cp_size, forward_batch,
9         torch.cuda.current_stream(),
10    )
11    attn_backend.store_cache(layer_id=self.layer_id, swa_k=kv, # 写
    完整 8 个 token
12    forward_batch=forward_batch)
13 else:
14     self._compute_kv_to_cache(...) # 非 CP: 融合直写
15     kv = None

```

步骤七: KV 跨 CP 组 AllGather (核心通信 ①)

每个 rank 只算了 2 个 token 的 KV, 但 attention 要看全部 8 个 token 的 KV。因为步骤一已经把总 token 数对齐成 `cp_size` 的倍数, 各 rank 的行数天然相等, 所以这里通信本身不需要再补齐形状, 直接 `new_empty` + `all_gather_into_tensor`:

```

</> cp_utils.py Python |

```

```

1 # cp_all_gather_rerange_output
2 if is_dsa_prefill_cp_round_robin_split():
3     output_tensor = input_tensor.new_empty(
4         (input_tensor.shape[0] * cp_size, *input_tensor.shape[1:]),
5     )
6     attn_cp_all_gather_into_tensor(output_tensor, input_tensor) #
    dp_attention.py:599
7     out_shape = output_tensor.shape
8     output_tensor = ( # 把「按 rank 分段」还原成「按
    token 自然序」
9         output_tensor.view(cp_size, -1, *out_shape[1:]))
10        .transpose(0, 1)
11        .reshape(out_shape)
12    )
13    return output_tensor

```

顺序还原示意：

</>

Plain Text

```

1 AllGather之前:
2 rank0: [t0,t4]
3 rank1: [t1,t5]
4 rank2: [t2,t6]
5 rank3: [t3,t7]
6 AllGather 之后 (按 rank 分段): [t0,t4 | t1,t5 | t2,t6 | t3,t7]
7 view(cp_size, -1) + transpose: [t0,t1,t2,t3, t4,t5,t6,t7] ← 自然序

```

gather 之后每个 rank 都有全部 8 个 token 的 KV，但 Q 仍只有自己的 2 个。

步骤八：写完整 KV cache，然后算 attention

写 cache：store_cache 要把 gather 来的 8 个 token 的 KV 写进 paged cache，需要知道「每个 token 写到第几个槽位」——这个槽位号来自 swa_out_cache_loc。它属于步骤四里没被切分的那批字段，所以有 8 个槽位号可用。反过来说：如果它跟着裁成 2 个，这里就只能写 2 个 token，cache 直接残缺——这就是那批字段必须留全局长度的原因。

</> python/sglang/srt/layers/attention/deepseek_v4_backend.py

Python

```

1 def store_cache(
2     self, layer_id: int, swa_k: torch.Tensor, forward_batch: ForwardBatch
3 ) -> None:
4     swa_loc = self.get_swa_out_cache_loc(forward_batch) # 获取全局的
    swa_out_cache_loc
5     if envs.SGLANG_OPT_USE_FUSED_STORE_CACHE.get():

```

```

6         self.token_to_kv_pool.set_swa_key_buffer_radix_fused(
7             layer_id=layer_id,
8             swa_loc=swa_loc,
9             cache_k=swa_k,
10        )
11    else:
12        if self.is_bf16_kv_cache:
13            swa_k_pack = split_nope_bf16_rope_bf16(swa_k)
14        else:
15            swa_k_pack = quant_to_nope_fp8_rope_bf16_pack_triton(swa_k)
16        self.token_to_kv_pool.set_swa_key_buffer_radix(
17            layer_id=layer_id,
18            swa_loc=swa_loc,
19            cache_nope_fp8_rope_bf16_pack=swa_k_pack,
20        )

```

算 attention：Q只有本 rank 的 2 行，KV 是完整 8 行，输出还是 2 行。KV 上一步已经凑齐，这里不跟别的卡通信，纯本地计算。

</> deepseek_v4.py

Python

```

1 # MQALayer.forward
2 o = attn_backend.forward(
3     q=attn_q,
4     k=attn_k,
5     v=attn_k,
6     layer=self.attn_mqa,
7     forward_batch=forward_batch,
8     compress_ratio=self.compress_ratio,
9     attn_sink=self.attn_sink,
10    save_kv_cache=save_kv_cache,
11 )

```

kernel 是 `flash_mla_with_kvcache` (`deepseek_v4_backend.py:1442-1477`),
长 prefill 走 `_forward_prefill_sparse` (`:1484`)。

</> deepseek_v4_backend.py

Python

```

1 # DeepseekV4AttnBackend.forward
2 # 长prefill
3 if forward_batch.forward_mode.is_extend_without_speculative() and (
4     q.shape[0] > _LARGE_INDEXER_QUERY_THRESHOLD
5     or envs.SGLANG_OPT_FLASHMLA_SPARSE_PREFILL.get()
6 ):
7     return self._forward_prefill_sparse(

```

```

8         q=q,
9         layer_id=layer_id,
10        compress_ratio=compress_ratio,
11        forward_batch=forward_batch,
12        token_to_kv_pool=token_to_kv_pool,
13        core_attn_metadata=core_attn_metadata,
14        attn_sink=attn_sink,
15    )
16 # 默认路径
17 import sgl_kernel.flash_mla as flash_mla
18
19 o = flash_mla.flash_mla_with_kvcache(
20     q=q,
21     k_cache=swa_k_cache,
22     head_dim_v=self.head_dim_v,
23     block_table=None,
24     cache_seqlens=None,
25     tile_scheduler_metadata=flashmla_metadata,
26     softmax_scale=self.softmax_scale,
27     is_fp8_kvcache=not self.is_bf16_kv_cache,
28     indices=swa_page_indices,
29     topk_length=swa_topk_lengths,
30     attn_sink=attn_sink,
31     extra_k_cache=extra_k_cache,
32     extra_indices_in_kvcache=extra_indices,
33     extra_topk_length=extra_topk_lengths,
34 )[0]
35 o = o.squeeze(1)
36 return o

```

值得注意的是：

- 多流 overlap 被显式关掉（`deepseek_v4.py:938`）。
 - 多流的做法是把算 Q 和算 KV 丢到两条 CUDA stream 上并行，但那条路径里 KV 是「算完顺手写进 cache」一步做完的
 - CP 需要「算完 → 跨卡 gather → 再写」，中间多一步，塞不进原来的融合 kernel。

</>

Python

```

1 enable_multi_stream = (
2     envs.SGLANG_OPT_USE_MULTI_STREAM_OVERLAP.get()
3     and self.alt_streams is not None
4     and get_is_capture_mode()
5     and x.shape[0] <= self._multi_stream_bs_limit

```

```

6     and not (self.dsa_enable_prefill_cp and
    dsa_use_prefill_cp(forward_batch)) # 不使用cp并行
7     and not (_is_hip and self.compressor is None)
8 )

```

步骤九：进 MoE 前 hidden AllGather（核心通信 ②，仅 a2a=None）

当a2a=None时使用的是FuseMOE的kernel，需要使用完整的 token 才能正确路由，但每个rank 只有 2 个 token 的 hidden，因此需要使用 `dsa_cp_gather_hidden_states` 获取完整的hidden_states

</> deepseek_v4.py

Python

```

1 # DeepseekV4DecoderLayer.forward
2 if _use_cp:
3     if get_moe_a2a_backend().is_none():
4         hidden_states = dsa_cp_gather_hidden_states(hidden_states)
5     else:
6         assert get_moe_a2a_backend().is_deepestp(), (
7             "CP requires DeepEP (moe_a2a_backend == deepestp). "
8             "Only DeepEP is tested with CP's per-rank token split."
9         )

```

`dsa_cp_gather_hidden_states` 在CP_Group内做all_gather，获取全量的hidden_state

</> communicator_dsa_cp.py

Python

```

1 def dsa_cp_gather_hidden_states(hidden_states: torch.Tensor):
2     attn_dp_size = get_attention_dp_size()
3     attn_tp_size = get_attention_tp_size()
4     assert attn_dp_size == 1 and attn_tp_size == 1 # round-robin-split的
    cp必须满足dp=1, tp_size == 1 待查
5     hidden_states, local_hidden_states = (
6         get_local_dp_buffer(get_attention_cp_group()), # 借一个全局
        buffer
7         hidden_states,
8     )
9     attn_cp_all_gather_into_tensor(hidden_states, local_hidden_states) #
    all_gather操作
10    return hidden_states # 现在是全 8 个
    token

```

步骤十：MoE forward

</>

```

1 # DeepseekV4DecoderLayer.forward
2 hidden_states = self.mlp(
3     hidden_states,
4     forward_batch,
5     input_ids=input_ids,
6     input_ids_global=input_ids_global,
7     use_reduce_scatter=use_cp
8 )

```

`use_reduce_scatter=use_cp=True` 的作用是跳过 MoE 内部的 AllReduce（步骤十一的 ReduceScatter 会一并完成求和，否则重复 reduce）。

注意它只对 `forward_normal` 路径生效；`forward_deepest` 根本不读这个参数 —— DeepEP 的 combine a2a 已经把数据放回原位。

</> deepseek_v2.py

Python

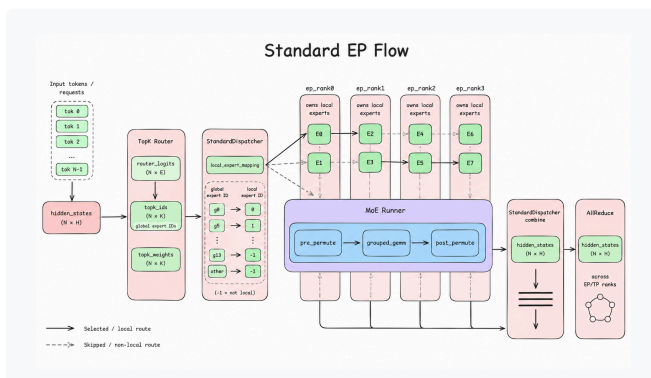
```

1 # DeepseekV2MOE.froward
2 if not self._enable_a2a_moe:
3     return self.forward_normal(
4         hidden_states,
5         should_allreduce_fusion,
6         use_reduce_scatter,
7         gemm_output_zero_allocator,
8         input_ids,
9         input_ids_global=input_ids_global,
10    )
11 else:
12     return self.forward_deepest(
13         hidden_states, forward_batch, input_ids_global=input_ids_global
14    )

```

1. forward_normal()

不做 All-to-All 通信，而是采用 AllGather 全量 token + 本地只算自己负责的专家 + AllReduce 归约的策略。



- 路由计算 (TopK Router)
 - 计算 Token 路由，输出全局专家 ID (topk_ids) 及其对应权重。
- 本地分发 (Dispatch)
 - 将全局专家 ID 映射为本地专家 ID（属于当前 Rank 的分配 Local ID，非本 Rank 的标记为 -1）。

特点

1. 无 **token 物理搬移**: dispatch 阶段不移动 token 数据, 只重映射 expert ID
2. 每个 **rank 处理全部 token**: 但只在本地的专家上计算, 其余跳过 (id=-1 的 token 贡献为 0)
3. 靠 **AllReduce 聚合**: 各 rank 的部分结果相加得到最终结果

• 专家执行 (MoE Runner)

- **重排 (Pre-permute)**: 按专家 ID 对 Token 排序, 直接过滤/跳过 ID 为 -1 的 Token。
- **计算 (Grouped GEMM)**: 仅对分配到本地的专家执行核心矩阵运算 ($W13 + W2$)。
- **还原 (Post-permute)**: 将计算后的 Token 恢复至原始输入顺序。

• 状态合并 (Combine)

- 输出当前 Rank 计算完成的隐藏层状态 (Hidden States)。

• 全局聚合 (AllReduce)

- 跨所有 EP Rank 执行通信 (AllReduce), 将各节点分散计算的结果相加, 得到最终完整输出。

2. forward_deepest()

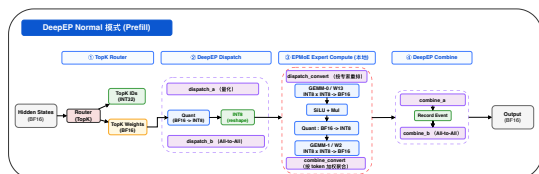
DeepEP 模式使用 **真正的 All-to-All 通信**: 将 token 物理发送到目标专家所在的 rank, 每个 rank 只接收和处理路由到自己的 token。

- buffer: 申请和管理通信 buffer + 持有通信上下文 + dispatch/combine kernel launcher
- handle: dispatch 阶段生成的路由描述符, combine 需要这个 handle

分为两种模式 Normal 和 LowLatency 模式

- prefill 使用 normal 模式
- decode 使用 low-latency 模式

Normal 模式 (Prefill/Extend)



dispatch_b 通信过程

- 需要先运行一个 **Layout kernel**, 扫描所有 token 的路由决策, 计算每个 rank 接收多少 token、每个 expert 接收多少 token, 再据此动态确定通信

• dispatch_a (量化 + 捕获事件)

- **作用**: 将 BF16 tensor 量化为 FP8, 捕获通信分发事件。

```
Python
1 def dispatch_a(self,
  hidden_states,
  topk_output):
2     topk_weights, topk_ids
  =
  topk_output.topk_weights,
  topk_output.topk_ids
```


偏移量。这引入了额外的 kernel launch 和同步。

```

3     topk_ids =
      topk_ids.to(torch.int64)
4     if
      deep_gemm_wrapper.ENABLE_J
      IT_DEEPGEMM and
      self.use_fp8:
5         # 返回 tuple: (fp8
      tensor, scale) 1. fp8 per-
      token 量化
6         hidden_states =
      sglang_per_token_group_qua
      nt_fp8(
7             hidden_states,
      128,
8
      column_major_scales=deep_g
      emm_wrapper.DEEPGEMM_SCALE
      _UE8M0,
9
      scale_tma_aligned=deep_gem
      m_wrapper.DEEPGEMM_SCALE_U
      E8M0,
10
      scale_ue8m0=deep_gemm_wrap
      per.DEEPGEMM_SCALE_UE8M0,
11     )
12     # 2. 捕获事件（用于异步
      overlap)
13     previous_event =
      Buffer.capture() if
      self.async_finish else
      None
14     return hidden_states,
      topk_ids, topk_weights,
      previous_event

```

- dispatch_b (all-to-all 通信)
 - 作用：执行 all-to-all 通信，将 token 分发到目标专家所在的 rank。

```

</> Python |
1 def dispatch_b(self,
      hidden_states, topk_ids,

```

```
topk_weights,
previous_event):
2     # 调用 _dispatch_core
    执行实际通信
3     (hidden_states,
topk_ids, topk_weights,
4
num_recv_tokens_per_expert
, event) =
self._dispatch_core(...)
5     # 等待通信完成
6
event.current_stream_wait(
) if self.async_finish
else ()
7     # 解包接收到的数据
8     if
    isinstance(hidden_states,
tuple):
9         hidden_states,
hidden_states_scale =
hidden_states
10     else:
11
hidden_states_scale = None
12
13     return
    DeepEPNormalDispatchOutput
(
14         hidden_states,
        # [num_recv_tokens,
hidden] FP8
15
hidden_states_scale,
        # [num_recv_tokens,
hidden//128] FP32/UE8M0
16         topk_ids,
        # [num_recv_tokens, topk]
INT64
17         topk_weights,
        # [num_recv_tokens, topk]
FP32
18
num_recv_tokens_per_expert
, # List[int], CPU 侧
```

19)

- 专家计算 (Normal contiguous)
- combine_a (准备回传)

仅传递, 无计算:

```
</> Python |
1 def combine_a(self,
  hidden_states, topk_ids,
  topk_weights):
2     if
    deep_gemm_wrapper.ENABLE_J
    IT_DEEPGEMM or _use_aiter
    or _is_npu:
3         output =
    hidden_states
4     else:
5         raise
    NotImplementedError() #
    triton runner 暂时禁用
6     previous_event =
    Buffer.capture() if
    self.async_finish else
    None
7     return output,
    previous_event
```

- combine_b (all-to-all 回传)

```
</> Python |
1 def combine_b(self,
  output, previous_event):
2     hidden_states, event =
    self._combine_core(output,
    previous_event)
3
    event.current_stream_wait(
    ) if self.async_finish
    else () # 通信同步
4     self.handle = None
    # 清理 dispatch 留下的状态
5     self.src2dst = None
6     return hidden_states
    # [num_original_tokens,
    hidden] BF16
```

```

7
8 def _combine_core(self, x,
previous_event):
9     combined_x, _, event =
buffer.combine(
10         x, self.handle,
# topk_weights 不作为参数传
入
11
async_finish=self.async_fi
nish,
12
previous_event=previous_ev
ent,
13
allocate_on_comm_stream=pr
evious_event is not None,
14
config=DeepEPConfig.get_in
stance().normal_combine_co
nfig,
15     )
16     return combined_x,
event

```

步骤十一：出 MoE 后 ReduceScatter 回收（核心通信 ③，仅 a2a=None）

</> deepseek_v4.py

Python

```

1 # DeepseekV4DecoderLayer.forward
2 if _use_cp and get_moe_a2a_backend().is_none():
3     hidden_states = dsa_cp_reduce_scatter_hidden_states(hidden_states)

```

</>

Python

```

1 # communicator_dsa_cp.py:67 dsa_cp_reduce_scatter_hidden_states
2 def dsa_cp_reduce_scatter_hidden_states(hidden_states: torch.Tensor):
3     attn_dp_size = get_attention_dp_size()
4     attn_tp_size = get_attention_tp_size()
5     assert attn_dp_size == 1 and attn_tp_size == 1
6     cp_size = get_attention_cp_size()
7     cp_rank = get_attention_cp_rank()
8     input_hidden_states = hidden_states

```

```

9     hidden_states = hidden_states.tensor_split(cp_size)[cp_rank]      #
    取自己那一段
10     attn_cp_reduce_scatter_tensor(hidden_states, input_hidden_states)  #
    边求和边切
11     return hidden_states                                              #
    回到 2 个 token

```

把「TP partial 求和」和「按 cp_rank 切片」融进同一个 ReduceScatter，比 AllReduce + slice 省一半通信量。

步骤十二：末层输出 AllGather 还原（核心通信 ④）

层循环跑完后必须把 4 个 rank 的 1/4 输出拼回完整序列，否则 `hc_head` / `norm` / `lm_head` 拿到的是残缺序列。

```

</> deepseek_v4.py Python |
1 # [CP] CP 收口: all-gather + 还原交错顺序
2 if self.pp_group.is_last_rank and dsa_use_prefill_cp(forward_batch):
3     hidden_states = cp_all_gather_rerange_output(
4         hidden_states, self.cp_size, forward_batch,
5         torch.cuda.current_stream(),
6     )

```

只在最后一个 PP rank 做，因为 PP 之间的 IPC 传的就是 CP 切分后的张量。之后 `hc_head` + `norm` + `LogitsProcessor` 都在完整序列上跑，与非 CP 完全一致。

附录

A. Ring-AllReduce通信

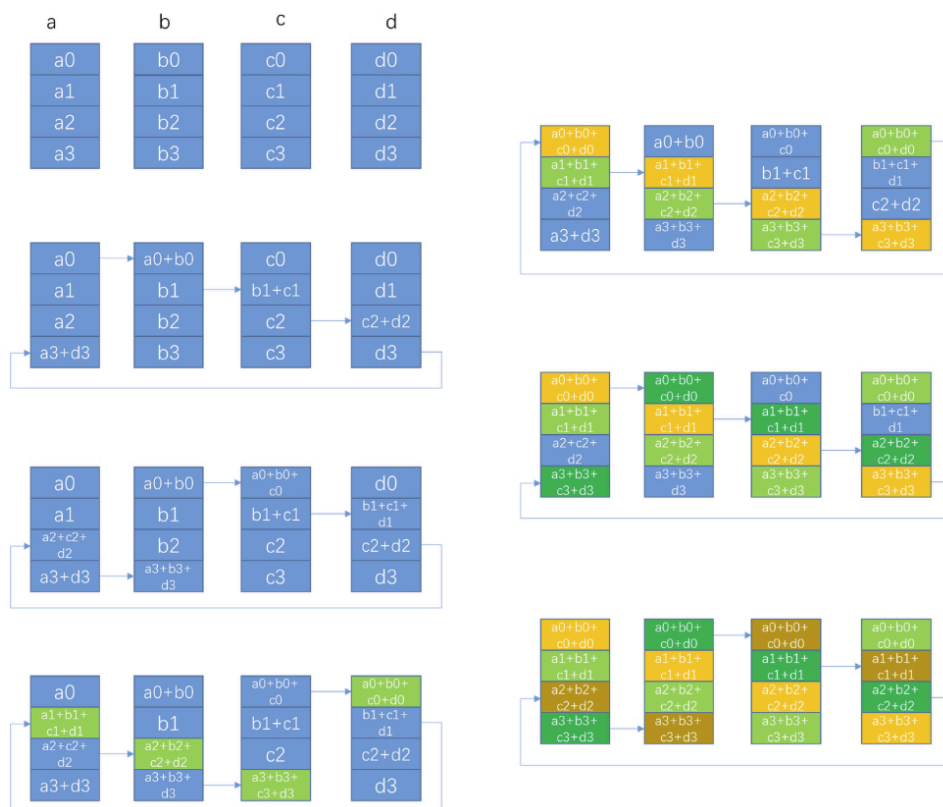
假设模型梯度大小是G，假设有N个dp_rank

AllReduce会将自己的这份梯度发送给所有的机器，也就是说，每个rank的通信量为NxG

Ring-AllReduce将自己的这份参数分成N份，每份的大小为G/N，然后分两阶段传输

1. Scatter-Reduce：每次每个rank发送G/N的大小，经过N-1次传输后，每个rank至少有一个计算完整的片段
 2. AllGather：接着将这个计算完整的片段，广播给每一个rank，这也需要经过N-1次传输
- 最终，Ring-AllReduce的传输方式在每个rank的通信量为： $2 \cdot (N-1) \cdot G/N$ ，当N足够大时，每个

rank的最多通信2G的参数，相比于AllReduce通信量显著下降。



B. SGLang 各种并行size和rank计算

下面是 parallel_state.py 的原始代码：

```
</> Python |

1 # ===== Attention 侧: 把 tp_size 预算切成 [attn_dp × attn_cp × attn_tp]
  =====
2 # 假设tp=4, pp=2, dp=2, ep=2, cp=1
3 # world_size = tp × pp = 4 × 2 = 8。dp=2 和 ep=2 是从 tp=4 里切出来的，不额外
  占卡。
4 attn_dp_size = attention_data_parallel_size # = dp_size (仅当
  enable_dp_attention=True)
5 # 否则 =1 (不开
  DP-Attention)
6 attn_cp_size = attention_context_model_parallel_size # = cp_size (序列
  并行，本例=1)
7 attn_tp_size = tensor_model_parallel_size // attn_cp_size // attn_dp_size
8 # ★ Attention 真
  正用的 TP 大小
9 # tp 预算先刨掉
  CP 份，再刨掉 DP 份，
10 # 剩下的才是
  attention 内部 TP 切分的份数
```

```

11 # 本例: 4 // 1
    // 2 = 2
12
13 # ===== MoE 侧: 把 tp_size 预算切成 [moe_dp × moe_ep × moe_tp] =====
14 moe_ep_size = expert_model_parallel_size # = ep_size (专家
    并行, 本例=2)
15 moe_dp_size = moe_data_model_parallel_size # MoE 的 DP 份
    (本例=1, 与 attn_dp 独立)
16 moe_tp_size = tensor_model_parallel_size // moe_ep_size // moe_dp_size
17 # ★ MoE 真正用的
    TP 大小
18 # tp 预算先刨掉
    EP 份, 再刨掉 moe_dp 份
19 # 本例: 4 // 2
    // 1 = 2

```

这两个公式是全文最关键的一行——它说明 Attention 和 MoE 走的是 **两套独立的坐标系统**，共享同一个 tp_size 预算。两套系统各自把 tp_size 拆成三个因子的乘积，互不影响：Attention 切 $[DP \times CP \times ATTN_TP]$ ，MoE 切 $[MOE_DP \times EP \times MOE_TP]$ ，只要两边乘积都等于 tp_size 即可。

Attention部分的并行布局： $DP \rightarrow CP \rightarrow ATTN_TP$ ，即 $tp_rank = (attn_dp_rank \times attn_cp_size + attn_cp_rank) \times attn_tp_size + attn_tp_rank$

```

</> Python |
1 # Attention 侧分解 (attn_cp_size=1 时简化为 DP → ATTN_TP 两层)
2 attn_tp_rank = tp_rank % attn_tp_size # 最
    低位: TP
3 attn_cp_rank = (tp_rank // attn_tp_size) % attn_cp_size # 中
    间位: CP
4 attn_dp_rank = tp_rank // (attn_tp_size * attn_cp_size) # 最
    高位: DP

```

MoE部分的并行布局： $MOE_DP \rightarrow EP \rightarrow MOE_TP$ ， $tp_rank = (moe_dp_rank \times moe_ep_size + moe_ep_rank) \times moe_tp_size + moe_tp_rank$

```

</> Python |
1 # MoE 侧分解
2 moe_tp_rank = tp_rank % moe_tp_size
3 moe_ep_rank = (tp_rank % (tp_size // moe_dp_size)) // (tp_size //
    moe_dp_size // moe_ep_size) # moe_ep_rank = tp_rank % moe_ep_size
4 moe_dp_rank = tp_rank // (tp_size // moe_dp_size)

```

套用公式计算各个卡上的rank如下表： $tp=4, pp=2, dp=2, ep=2, cp=1$

1	全局 rank	pp_rank	tp_rank	attn_dp	attn_tp	moe_dp	moe_ep	moe_tp
2	0	0	0	0	0	0	0	0
3	1	0	1	0	1	0	0	1
4	2	0	2	1	0	0	1	0
5	3	0	3	1	1	0	1	1
6	4	1	0	0	0	0	0	0
7	5	1	1	0	1	0	0	1
8	6	1	2	1	0	0	1	0
9	7	1	3	1	1	0	1	1

C. DSV4 TP+DP+PP+EP混合并行

1 整体结构

</>

Plain Text

|

```
1 DeepseekV4ForCausalLM
2 |— embed_tokens:      VocabParallelEmbedding      ← 词表切分
3 |— layers[0..42]:     DeepseekV4DecoderLayer x43
4 |   |— input_layernorm
5 |   |— self_attn:      MQALayer                    ← Attention (本章重点)
6 |   |— post_attn_layernorm
7 |   |— mHC 混合        ← 替代普通残差, 本报告略
8 |   |— mlp:            DeepseekV2MoE                ← MoE (本章重点)
9 |   |— mHC 混合
10 |— norm
11 |— lm_head:           ParallelLMHead               ← 词表切分 + AllGather
```

PP 下: `embed_tokens` + 前若干层在 stage 0, 后若干层 + `norm` + `lm_head` 在最后一个 stage, 中间 stage 用 `PPProxyTensors` 传 hidden。

2 MQALayer: Attention 的 6 个权重

DSV4 用 **MQA (Multi-Query Attention)** + **LoRA** 压缩, 比 DSV2 的 MLA 更激进。一条 forward 路径如下 ([deepseek_v4.py:936-1111](#)):

1. 前置: 四种并行线性层

在看 forward 之前，先搞懂 SGLang 里四种 `ParallelLinear` 的切分与通信方式 (`linear.py`)。它们是 TP 的基本积木，DSV4 的每个权重都是其中之一。每个类有三段关键逻辑：`__init__`（算权重 shape）、`weight_loader`（加载时切权重）、`forward`（前向 + 通信）。下面把三段都贴全。

(1) ColumnParallelLinear —— 列切，forward 不通信

定义 $Y = X @ A + b$ ，把 `A` 沿输出维（dim=1）切，每个 rank 持有一列块。

`__init__` (`linear.py:315-380`) —— 算出每 rank 的权重 shape：

```
</> Python |
1 class ColumnParallelLinear(LinearBase):
2     def __init__(self, input_size, output_size, ..., gather_output=False,
3         ...):
4         # 输出维按 tp 切: 每 rank 只持有 output_size/tp_size 列
5         self.output_size_per_partition = divide(self.output_size,
6             tp_size)
7         self.output_partition_sizes = [self.output_size_per_partition]
8         # 权重实际 shape = [input_size, output_size_per_partition]
9         # bias 也按 tp 切: [output_size_per_partition]
10        if bias:
11            self.bias =
12            Parameter(torch.zeros(self.output_size_per_partition, ...))
```

`weight_loader` (`linear.py:384-437`) —— checkpoint 是完整权重，每个 rank 只 copy 自己那一段：

```
</> Python |
1 def weight_loader(self, param, loaded_weight):
2     output_dim = getattr(param, "output_dim", None) # 输出维的索引（通
3     常是 0 或 1）
4     if output_dim is not None:
5         shard_size = param_data.shape[output_dim] # 每 rank 的列
6         块大小
7         start_idx = self.tp_rank * shard_size # * 本 rank 从
8         哪开始切
9         # 从完整权重里 narrow 出 [start_idx : start_idx+shard_size] 这一
10        段
11        loaded_weight = loaded_weight.narrow(output_dim, start_idx,
12            shard_size)
13        param_data.copy_(loaded_weight) # 拷到本 rank
14        的参数里
```

`forward` (`linear.py:466-478`) —— 各 rank 独立算，不 AllReduce：

</>

Python

```

1  def forward(self, input_):
2      bias = self.bias if not self.skip_bias_add else None
3      output_parallel = self.quant_method.apply(self, input_, bias) #
    Y_i = X @ A_i
4      if self.gather_output:
5          # 可选: 把各 rank 的列块 all_gather 拼回完整 Y
6          output = tensor_model_parallel_all_gather(output_parallel)
7      else:
8          output = output_parallel # 默认: 不拼, 每 rank 留自己那份
9      return output

```

</>

Plain Text

```

1  A = [ A_0 | A_1 | A_2 | A_3 ]      ← 沿输出维(dim=1)切 4 份 (weight_loader
    做)
2      GPU0    GPU1    GPU2    GPU3
3  Y_i = X @ A_i    (forward: 各算各的, X 完整复制, 不通信)

```

关键: 列切的 forward 天然不通信, 因为各 rank 算的是输出的不同列, 互不重叠。只有 `gather_output=True` 时才 `all_gather` 拼回。

(2) RowParallelLinear —— 行切, forward 必须 AllReduce

把 `A` 沿输入维 (`dim=0`) 切成纵向堆叠, `X` 沿输出维 (`dim=1`) 切成 `X = [X_1, ..., X_p]`。

`__init__` ([linear.py:1365-1424](#)) —— 输入维按 `tp` 切, 多了 `reduce_results` 开关:

</>

Python

```

1  class RowParallelLinear(LinearBase):
2      def __init__(self, input_size, output_size, ...,
3                  input_is_parallel=True, reduce_results=True,
4                  use_dp_attention_reduce=False):
5      self.input_is_parallel = input_is_parallel
6      self.reduce_results = reduce_results # * 是否在
    forward 里 AllReduce
7      self.use_dp_attention_reduce = use_dp_attention_reduce # * 用
    attn_tp 组还是 tp 组
8      # 输入维按 tp 切: 每 rank 只持有 input_size/tp_size 行
9      self.input_size_per_partition = divide(input_size, self.tp_size)
10     # 权重 shape = [input_size_per_partition, output_size]
11     # bias 不切: [output_size] (因为输出是完整的, 只是部分和)
12     if bias:

```

```
13         self.bias = Parameter(torch.zeros(self.output_size, ...))
```

weight_loader ([linear.py:1426-1470](#)) —— 沿 输入维 narrow, 和列切对称:

</>

Python

```
1     def weight_loader(self, param, loaded_weight):
2         input_dim = getattr(param, "input_dim", None)      # 输入维的索引
3         if input_dim is not None:
4             shard_size = param_data.shape[input_dim]      # 每 rank 的行
块大小
5             start_idx = self.tp_rank * shard_size          # ★ 本 rank 从
哪开始切
6             loaded_weight = loaded_weight.narrow(input_dim, start_idx,
shard_size)
7             param_data.copy_(loaded_weight)
```

forward ([linear.py:1519-1563](#)) —— 核心通信点在这里:

</>

Python

```
1     def forward(self, input_, skip_all_reduce=False, forward_batch=None):
2         # 1) 输入处理: input_is_parallel=True 时输入已是切好的; 否则自己切
3         if self.input_is_parallel:
4             input_parallel = input_                        # 上游
ColumnParallel 已切好
5         else:
6             splitted = split_tensor_along_last_dim(input_,
num_partitions=self.tp_size)
7             input_parallel = splitted[self.tp_rank].contiguous()
8
9         # 2) 矩阵乘: Y_i = X_i @ A_i (部分和, bias 只在 rank 0 加, 避免重复)
10        bias_ = None if (self.tp_rank > 0 or self.skip_bias_add) else
self.bias
11        output_parallel = self.quant_method.apply(self, input_parallel,
bias=bias_)
12
13        # 3) ★ AllReduce ★ (受 reduce_results 开关控制)
14        if self.reduce_results and self.tp_size > 1 and not
skip_all_reduce:
15            if self.use_dp_attention_reduce:
16                # DP-Attention: 只在更小的 attn_tp_group 上 reduce
17                output =
get_attention_tp_group().all_reduce(output_parallel)
18            else:
19                # 普通 TP: 在整个 tp_group 上 reduce
```

```

20         output =
    tensor_model_parallel_all_reduce(output_parallel)
21     else:
22         output = output_parallel          # 不 reduce, 返回部分和 (让
        外层自己处理)
23     return output, output_bias

```

</>

Plain Text

```

1      [ A_0 ]          X = [ X_0 | X_1 | X_2 | X_3 ]
2  A = [ A_1 ]          Y_i = X_i @ A_i (部分和)
3      [ A_2 ]          Y = Σ Y_i  ←★ AllReduce (forward 第 3 步) ★
4      [ A_3 ]

```

记忆口诀：列切不通信（输出天然分块），行切必 AllReduce（部分和要汇总）。TP 链路里出现 AllReduce 的地方，前面一定有个 `reduce_results=True` 的 `RowParallelLinear`。

`reduce_results` 为什么要有开关：有时候外层想把多个 `RowParallel` 的部分和先在本地加起来，再一次性 AllReduce（省通信次数）。DSV4 的 MoE 就是这么做的——shared expert 和 routed expert 的 `down_proj` 都设 `reduce_results=False`，partial 相加后统一 reduce（见 3.3 节）。

(3) MergedColumnParallelLinear —— 打包的列切，forward 不通信

继承自 `ColumnParallelLinear`，把多个列切矩阵打包成一个。典型用途：把 `gate_proj` 和 `up_proj`（都是列切）拼成一个 `gate_up_proj`，一次 GEMM 算完两个，省一次 kernel launch。

`__init__` ([linear.py:512-547](#)) —— 接收 `output_sizes` 列表，每块独立按 tp 切：

</>

Python

```

1 class MergedColumnParallelLinear(ColumnParallelLinear):
2     def __init__(self, input_size, output_sizes: List[int], ...):
3         # output_sizes e.g. [2048, 2048] for gate+up
4         self.output_sizes = output_sizes
5         # 每块必须能被 tp_size 整除
6         assert all(output_size % tp_size == 0 for output_size in
            output_sizes)
7         # 调父类: output_size = sum(output_sizes) = 4096, 整体当一个大列切矩阵
8         super().__init__(input_size=input_size,
            output_size=sum(output_sizes), ...)
9         # 父类会算 output_size_per_partition = 4096/tp_size
10        # 但这里重写 output_partition_sizes, 让每块各自记录切分后的大小
11        self.output_partition_sizes = [

```

```

12         divide(output_size, tp_size) for output_size in
    self.output_sizes
13     ] # e.g. [1024, 1024] for tp=2

```

weight_loader ([linear.py:549-650](#)) —— 比 ColumnParallel 复杂，要按 **shard_id** 分别切每块：

```

</> Python |
1  def weight_loader(self, param, loaded_weight, loaded_shard_id=None):
2      # ... (省略 GGUF / 量化特例)
3      assert loaded_shard_id < len(self.output_sizes)
4      if output_dim is not None:
5          # ★ 关键: shard_offset 是本块在打包矩阵里的起始位置 (已按 tp 缩放)
6          shard_offset = sum(self.output_sizes[:loaded_shard_id]) //
self.tp_size
7          shard_size = self.output_sizes[loaded_shard_id] //
self.tp_size
8          # e.g. gate(shard_id=0): offset=0, size=1024
9          #      up (shard_id=1): offset=1024, size=1024
10
11         param_data = param_data.narrow(output_dim, shard_offset,
shard_size)
12         start_idx = self.tp_rank * shard_size # 本 rank 在本
块内的起点
13         loaded_weight = loaded_weight.narrow(output_dim, start_idx,
shard_size)
14         param_data.copy_(loaded_weight)

```

forward —— 直接继承 **ColumnParallelLinear.forward**，不重写。一次 GEMM 算完所有打包块，输出是拼接的：

```

</> Python |
1  # 等同于 ColumnParallelLinear.forward:
2  # output = X @ [gate_i | up_i] → 一次 GEMM 出 [gate_output,
up_output]
3  # 不通信 (除非 gather_output=True)

```

```

</> Plain Text |
1  gate_up 权重 (完整) = [ gate | up ]          ← 两块沿输出维拼一起
2                        2048   2048
3
4  每 rank 持有 (tp=2) = [ gate_i | up_i ]      ← 每块各自列切后拼一起
5                        1024   1024

```

```

6          \-----/
7          GPU0 的一份
8
9 forward: Y = X @ [gate_i | up_i] → 一次 GEMM, 输出 [Y_gate, Y_up] 拼接

```

和普通 **ColumnParallel** 的两点区别:

1. 权重 **shape**: `output_partition_sizes` 是列表 (每块各算), 不是单个值。量化方法 (如 FP8) 需要知道每块边界才能正确做 group quant。
2. **weight_loader** 多了 `shard_id`: 加载 gate 权重时 `shard_id=0`, 加载 up 权重时 `shard_id=1`, loader 根据 `shard_id` 算出正确的 offset, 把每块放到打包矩阵的正确位置。普通 **ColumnParallel** 没有这个参数。

forward 完全一样, 都是列切不通信。

(4) **ReplicatedLinear** —— 不切, 每 rank 完整复制, forward 不通信

最简单的一种: 权重不切分, 每个 rank 持有完整的 `[input_size, output_size]` 权重, 各算各的, 输入输出都完整。DSV4 的 `wq_a` / `wkv` (LoRA 下投影) 和 NextN 的 `e_proj` / `h_proj` 都是这种。

`__init__` ([linear.py:194-251](#)) —— 不做任何 divide, 权重和 bias 都是完整尺寸:

```

</> Python |
1 class ReplicatedLinear(LinearBase):
2     def __init__(self, input_size, output_size, ..., bias=True, ...):
3         # 不切! 直接用完整 input_size / output_size 创建权重
4         self.quant_method.create_weights(
5             self,
6             self.input_size,          # 完整, 不除 tp_size
7             [self.output_size],       # 完整, 不除 tp_size
8             ...
9         )
10        if bias:
11            self.bias = Parameter(torch.empty(self.output_size, ...)) #
完整 bias

```

weight_loader ([linear.py:253-275](#)) —— 直接 copy 全量, 不 narrow:

```

</> Python |
1 def weight_loader(self, param, loaded_weight):
2     # 没有 start_idx / shard_size / narrow -- 整个权重直接拷
3     assert param.size() == loaded_weight.size()

```

```
4      param.data.copy_(loaded_weight)      # 每个 rank 都 copy 完整
      权重
```

forward ([linear.py:277-282](#)) —— 普通 Linear，无任何集合通信：

```
</> Python |
1      def forward(self, x):
2          bias = self.bias if not self.skip_bias_add else None
3          output = self.quant_method.apply(self, x, bias)      # Y = X @ A +
      b, 完整计算
4          return output, output_bias      # 无
      all_reduce / all_gather
```

```
</> Plain Text |
1 A = [ 完整权重 ]      每个 rank 都有一份一模一样的 A
2 GPU0: A      GPU1: A      GPU2: A      GPU3: A      (4 份冗余)
3 Y = X @ A      (各算各的，输入输出都完整，不通信)
```

为什么要有 **ReplicatedLinear**：当某层的输出维度很小（如 DSV4 的 `wkv` 输出只有 512），切了反而通信开销大于计算开销；或者当该层的结果需要每个 rank 都完整拥有时（如 router gate，所有 rank 要算出同样的 topk 路由），就用复制不切。代价是冗余存储 + 冗余计算，但省了通信。

`python/sglang/srt/layers/linear.py` 还有很多 Linear 的变体可以自行查阅

2. forward 路径

```
</> Plain Text |
1 组网逻辑
2  _____
3 hidden (full, replicated)
4 |
5 |— wqkv_a (ReplicatedLinear)      ← 不切，每人都算完整低秩投影
6 |   q_a = hidden @ wqkv_a[:q_lora_rank]      → [tokens, 1024]
7 |   kv = hidden @ wqkv_a[q_lora_rank:]      → [tokens, 512]
8 |
9 |— q_norm / kv_norm (RMSNorm)      ← 不切
10 |
11 |— wq_b (ColumnParallelLinear)      ← 列切: n_heads × head_dim 沿 head 维切
```



```

12 |      q = q_a @ wq_b          → 每个 attn_tp rank 持有
    |      n_heads/attn_tp 个 head
13 |
14 | └─ [Attention kernel]        ← 不发起集合通信! 各 rank 算自己的 head
15 |      flash_mla_with_kvcache(...) → o = [tokens, local_heads ×
    |      head_dim]
16 |
17 | └─ wo_a (ColumnParallelLinear) ← 列切: n_groups × o_lora_rank 沿
    |      group 维切
18 |      o_compressed = o @ wo_a    → [tokens, local_groups ×
    |      o_lora_rank]
19 |
20 | └─ wo_b (RowParallelLinear)    ← 行切: 输入维切, ★ AllReduce ★
21 |      output = o_compressed @ wo_b
22 |      if attn_tp_size < global_tp_size:
23 |          output = attn_tp_all_reduce(output) ← DP-Attention 时显式
    |      reduce

```

</>

Plain Text

1 forward 路径	代码位置
2	_____
3 └─ wqkv_a	_forward_prepare() :763
4 qkv_a, _ = self.wqkv_a(x)	← ReplicatedLinear
5 q_lora = qkv_a[..., :1024]	:764 切出 q_a
6 kv = qkv_a[..., 1024:]	:795 切出 kv
7	
8 └─ q_norm	_forward_prepare() :791
9 q_lora = self.q_norm(q_lora)	← RMSNorm
10	
11 └─ wq_b	_forward_prepare() :792
12 q, _ = self.wq_b(q_lora)	← ColumnParallelLinear (列切)
13	
14 └─ [Attention kernel]	forward() :1058
15 o = attn_backend.forward(q, k, v, ...)	← flash_mla_with_kvcache
16	
17 └─ wo_a	forward() :1104-1105
18 wo_a = self.wo_a.weight.view(n_local_groups, o_lora_rank, -1)	
19 o = torch.einsum("tgd,grd->tgr", o, wo_a)	← 列切, 手动 einsum
20	
21 └─ wo_b + all_reduce	forward() :1107-1109
22 o, _ = self.wo_b(o.flatten(1))	← RowParallelLinear (行切)
23 if self.tp_size > 1 and self.tp_size < global_tp_size:	
24 o = attn_tp_all_reduce(o)	← DP-Attention 显式 reduce

关键数字 (DSV4-Flash 配置, [configs/deepseek_v4.py](#)):

1	权重	完整 shape	TP=2 时每卡 shape	切分方式
2	wqkv_a	[4096, 1536]	[4096, 1536]	不切 (replicated)
3	wq_b	[1024, 32768]	[1024, 16384]	列切 (head 维 / attn_tp)
4	wo_a	[32768, 1024]	[16384, 512]	列切 (group 维 / attn_tp)
5	wo_b	[1024, 4096]	[512, 4096]	行切 (输入维 / attn_tp)

Attention 的 TP 通信点 = 1 次 AllReduce, 在 `wo_b` ([deepseek_v4.py:1107](#))。Attention kernel 本身 不发起任何集合通信——它只算自己那份 head。

3. 每个权重的定义代码对照

上面 forward 路径里每个权重对应的定义在 [deepseek_v4.py:418-478](#)。注意它们用的 `tp_rank` / `tp_size` 是 `attn_tp_rank` / `attn_tp_size` (不是全局 tp), 这就是 DP-Attention 下 attention 只在小 attn_tp 组内切分的根源:

</>

Python

|

```
1 # wqkv_a: 融合的 q_a + kv 低秩投影, 复制不切
2 self.wqkv_a = ReplicatedLinear(
3     self.hidden_size,
4     self.q_lora_rank + self.head_dim,      # 1024 + 512 = 1536
5     ...
6 )
7
8 # wq_b: 列切, 沿 n_heads x head_dim 输出维切
9 self.wq_b = ColumnParallelLinear(
10     self.q_lora_rank,                      # 输入 1024 (不切)
11     self.n_heads * self.head_dim,          # 输出 32768 (按 attn_tp 切)
12     tp_rank=attn_tp_rank, tp_size=attn_tp_size, # * 用 attn_tp, 不是全局
13     tp
14     ...
15 )
16
17 # wo_a: 列切, 沿 n_groups x o_lora_rank 输出维切
18 self.wo_a = ColumnParallelLinear(
19     self.n_heads * self.head_dim // self.n_groups, # 输入 (不切)
20     self.n_groups * self.o_lora_rank,               # 输出 (按 attn_tp
21     切)
22     tp_rank=attn_tp_rank, tp_size=attn_tp_size,
```

```

21     ...
22 )
23
24 # wo_b: 行切, ★ reduce_results 有条件 ★
25 self.wo_b = RowParallelLinear(
26     self.n_groups * self.o_lora_rank,      # 输入 (按 attn_tp 行切)
27     self.hidden_size,                      # 输出 4096 (不切)
28     reduce_results=(attn_tp_size ==
29                     get_tensor_model_parallel_world_size()
30                     and attn_tp_size > 1), # ★ 条件见 3.2.4
31     tp_rank=attn_tp_rank, tp_size=attn_tp_size,
32     ...
33 )

```

4. 为什么 wo_b 的 AllReduce 是有条件的？为什么 DP-Attention 要显式 all_reduce？

这是全文最容易绕晕的点，对照代码讲清楚。涉及两段代码：

第一段：wo_b 定义时的 `reduce_results` 条件 ([deepseek_v4.py:473](#))

```

</> Python |
1 reduce_results = (attn_tp_size == get_tensor_model_parallel_world_size()
2                     and attn_tp_size > 1)
3 # get_tensor_model_parallel_world_size() = 全局 tp_size

```

- 纯 TP (`attn_tp_size == 全局 tp_size`)：条件为真 →
`reduce_results=True` → `wo_b` 内部走 `tensor_model_parallel_all_reduce`，在整个 `tp_group` 上 AllReduce。
- DP-Attention (`attn_tp_size < 全局 tp_size`)：条件为假 →
`reduce_results=False` → `wo_b` 内部 不做任何 reduce，返回部分和。

第二段：forward 里的显式 all_reduce ([deepseek_v4.py:1107-1109](#))

```

</> Python |
1 o, _ = self.wo_b(o.flatten(1)) # wo_b 内部不 reduce (DP-Attn 情况)
2 if self.tp_size > 1 and self.tp_size <
   get_tensor_model_parallel_world_size():
3     o = attn_tp_all_reduce(o) # ★ 在更小的 attn_tp_group 上显式 reduce
4 # 注: 这里的 self.tp_size 是 MQALayer 里设的 attn_tp_size

```

为什么不能让 wo_b 内部直接 reduce？关键在于 RowParallelLinear 内部的 reduce 用的是 `tensor_model_parallel_all_reduce`，它作用在整个 `tp_group` 上。但 DP-Attention 下，不同 `attn_dp rank` 处理的是不同的请求：

</>

Plain Text

- 1 纯 TP: 4 个 rank 处理同一请求 → 全 tp_group AllReduce 正确
- 2 [GPU0, GPU1, GPU2, GPU3] 同一请求 → reduce OK
- 3
- 4 DP-Attention: 2 个 attn_dp 组处理不同请求 → 全 tp_group AllReduce 会把不同请求混在一起!
- 5 [GPU0, GPU1] 请求A [GPU2, GPU3] 请求B
- 6 全 tp_group reduce = 把 A 和 B 的部分和相加 → 错误!
- 7 正确做法: 只在 [GPU0,GPU1] 内 reduce (A 自己), 只在 [GPU2,GPU3] 内 reduce (B 自己)

所以策略是: 关掉 `wo_b` 的内部全组 reduce, 改用 `attn_tp_all_reduce` 只在 `attn_tp_group` (同请求的小组) 内 reduce。这就是"显式 all_reduce"的由来——不是因为代码写法不同, 而是 通信域从小了一圈: 从 `tp_group` (全局) 缩到 `attn_tp_group` (同 DP 内的 TP 组)。

一句话: `reduce_results` 的条件本质是问"attention 的 TP 组是否就是全局 TP 组"。是 → 内部 reduce 省事; 否 → 必须改用更小的 `attn_tp` 组 reduce, 否则会把不同 DP 请求的数据混算。

3 DeepseekV2MoE (DSV4复用之前的结构): MoE 的两条路

在 `deepseek_v2.py` 中, MoE 有 两条 `forward` 路径, 取决于是否启用 A2A backend (DeepEP 等):

路径 A: `forward_normal` (纯 TP / TP+DP, 无 EP)

</>

Plain Text

- 1 组网逻辑
- 2 hidden (replicated)
- 3 |
- 4 | — shared_expert (TP 切分, `reduce_results=False`) → shared_partial
- 5 | gate_up = MergedColumnParallelLinear ← 中间维 / moe_tp
- 6 | down = RowParallelLinear ← 输入维 / moe_tp
- 7 |
- 8 | — gate (router, nn.Parameter, replicated) → router_logits [tokens, 256]
- 9 | 所有 rank 算出同样的路由 (因为 gate 复制 + 输入相同)
- 10 |
- 11 | — topk → 选 6 个专家
- 12 |
- 13 | — experts (FusedMoE, TP 切分)

```
14 | w13 = MergedColumnParallelLinear ← 每个 expert 的中间维 / moe_tp
15 | w2  = RowParallelLinear          ← 每个 expert 的输入维 / moe_tp
16 | → routed_partial
17 |
18 | └─ 本地融合 (无通信):
19 |     final = routed_scale × routed_partial + shared_partial
20 |
21 | └─ ★ tensor_model_parallel_all_reduce(final) ★ ← 唯一通信点
22 |     [deepseek_v2.py:1140]
```

</>

Plain Text

```
1 forward 路径                                代码行
2 _____
3 └─ shared_expert                             :1052
4 |     shared_output = self._forward_shared_experts(hidden_states)
5 |
6 └─ gate (router)                             :1056
7 |     router_logits = self.gate(hidden_states, ...)
8 |
9 └─ topk → 选 6 个专家                        :1062
10 |     topk_output = self.topk(hidden_states, router_logits, ...)
11 |
12 └─ experts (routed, TP 切分)                 :1104
13 |     final_hidden_states = self.experts(hidden_states, topk_output)
14 |
15 └─ 本地融合 (routed_scale × routed + shared) :1128
16 |     final = maybe_fuse_routed_scale_and_shared_add(
17 |         self.experts, final, shared_output, routed_scaling_factor)
18 |
19 └─ ★ AllReduce ★                             :1140
20     final = tensor_model_parallel_all_reduce(final)
```

纯 TP 下 MoE 的 TP 通信点 = 1 次 AllReduce。这是经典的 "shared + routed 都产 partial → 本地相加 → 单次 AllReduce" 融合，比各自 AllReduce 少一次通信。

路径 B: forward_deepep (启用 EP + DeepEP AllToAll)

</>

Plain Text

```
1 组网逻辑
2 hidden (replicated)
3 |
4 └─ shared_expert (★ TP1, 复制不切 ★) → shared_output (完整)
```

```

5 |     _shared_expert_use_tp1=True [deepseek_v2.py:705]
6 |
7 | └─ gate (replicated) → router_logits
8 | └─ topk → 选 6 个专家 (全局 expert id)
9 |
10 | └─ ★ dispatch AllToAll ★ ← 把 token 发给"持有目标 expert 的 rank"
11 |     [deepseek_v2.py:1483-1498]
12 |
13 | └─ experts 本地计算 (每个 rank 只有 256/ep 个完整 expert)
14 |     w13/w2 不切中间维, expert 整体持有
15 |
16 | └─ ★ combine AllToAll ★ ← 把结果发回源 rank 并累加
17 |     [deepseek_v2.py:1505-1517]
18 |
19 | └─ final = combined + shared_output ← 无 AllReduce!

```

</>

Plain Text

1 forward 路径	代码行
3 └─ shared_expert (★ TP1, 复制不切 ★)	:1225 或 :1229
4 shared_output = self._forward_shared_experts(hidden_states)	
5	
6 └─ gate (router)	:1220
7 router_logits = self.gate(hidden_states,	
8 forward_batch=forward_batch)	
9 └─ topk → 选 6 个专家 (全局 expert id)	:1235
10 topk_output = self.topk(hidden_states, router_logits, ...)	
11	
12 └─ experts (内部含 dispatch + 计算 + combine)	:1397
13 final_hidden_states = self.experts(hidden_states, topk_output)	
14	
15 └─ ★ dispatch AllToAll ★	← 在 FusedMoE.forward 内部调 dispatcher
16 self.experts.dispatcher.dispatch_a/dispatch_b	
17 (TBO 分解版在 :1483-1498)	
18	
19 └─ experts 本地计算	← run_moe_core (每个 rank 只有 256/ep 个 expert)
20	
21 └─ ★ combine AllToAll ★	← 在 FusedMoE.forward 内部调 dispatcher
22 self.experts.dispatcher.combine_a/combine_b	

```
23 | (TBO 分解版在 :1505-1517)
24 |
25 |—— 本地融合 (shared + routed) :1410-1418
26 | x = shared_output
27 | x.add(final_hidden_states, alpha=routed_scaling_factor)
28 | final_hidden_states = x
29 |
30 |—— (无 AllReduce! ) ← 函数在 :1425 直接 return
```

EP 下 MoE 的通信 = 2 次 AllToAll (dispatch + combine)，替代了 1 次 AllReduce。

shared expert 的两种模式（关键！）

1	模式	触发条件	shared expert 行为
2	TP 切分	无 A2A backend (纯 TP)	和 routed 一样切分，产 partial，在 AllReduce 前本地相加
3	TP1 复制	有 A2A backend (DeepEP 等)	不切分，每个 rank 持有完整 shared，在 AllToAll combine 后相加

4 embed_tokens 与 lm_head

</> Plain Text |

```
1 embed_tokens: VocabParallelEmbedding
2   - 无 DP-Attention: 词表沿 dim0 切 / tp_size, 每人持有 vocab/tp 个 token
  embedding
3   - 有 DP-Attention: enable_tp=False, 复制 (每个 DP rank 都有完整词表)
4
5 lm_head: ParallelLMHead (继承 VocabParallelEmbedding)
6   - 沿词表维切, 每卡算 vocab/tp 行 logits
7   - LogitsProcessor 内部 all_gather 拼回完整 logits
  [logits_processor.py:926]
```

D. 通信原语汇总

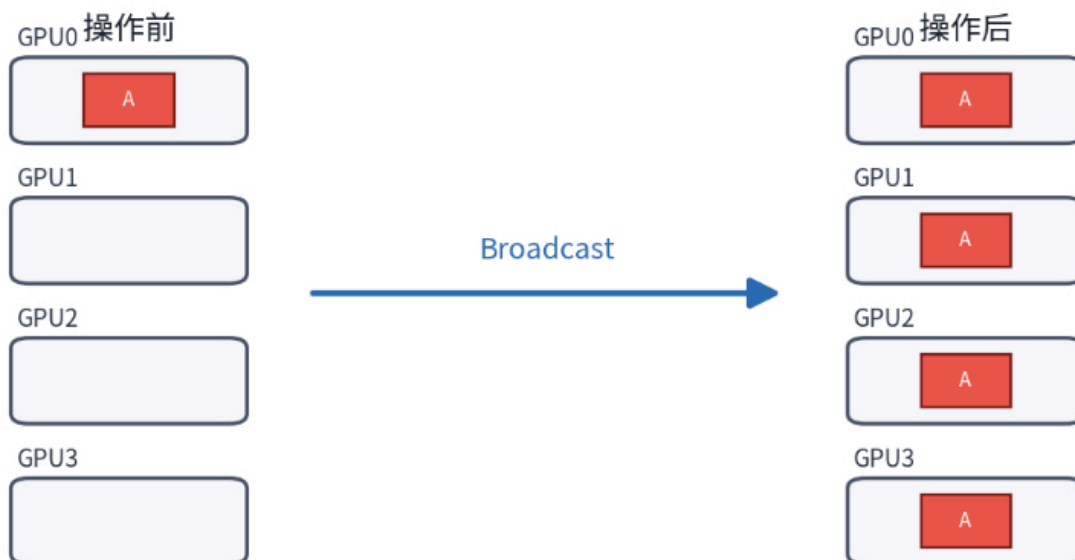
1. 1对多的基础原语

Broadcast（广播，1对多）

根节点把同一份完整数据复制到所有 GPU。典型用途：数据并行时把初始参数/权重从主卡广播到各卡，保证初值一致。

Broadcast 广播

1 对多：根节点把同一份完整数据复制到所有 GPU



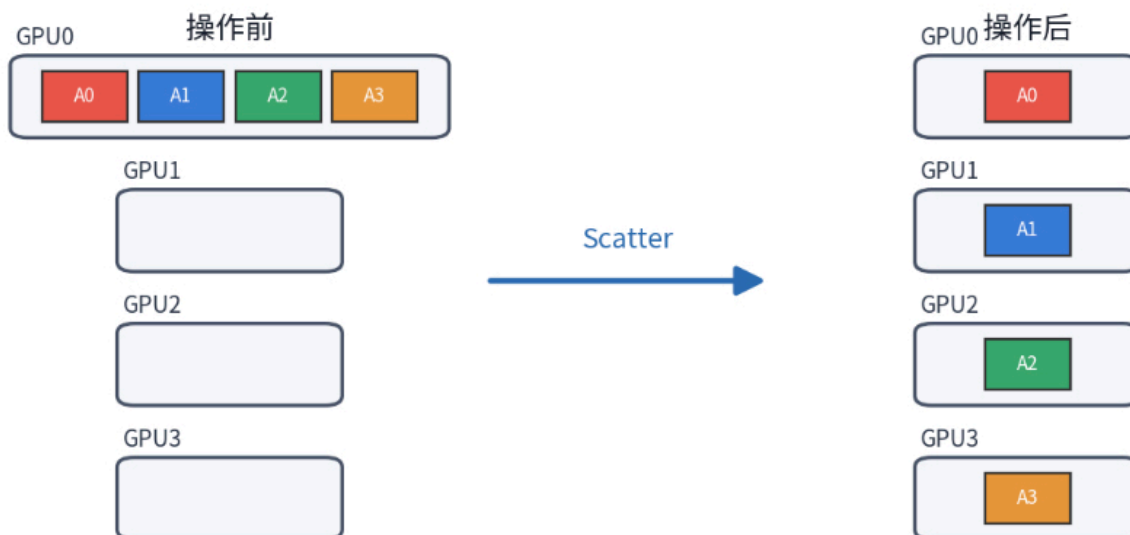
根 GPU0 的数据 A 被完整复制到每一张 GPU；常用于数据并行的参数/权重初始化，保证各卡初值一致。

Scatter（发散，1对多）

根节点把一个数组切成 N 份，第 i 份发给 GPU i 。与 Broadcast 的区别：各卡收到的是不同分片，而非完整副本。

Scatter 发散

1 对多：根节点把数据切成 N 份，第 i 份发给 GPU i



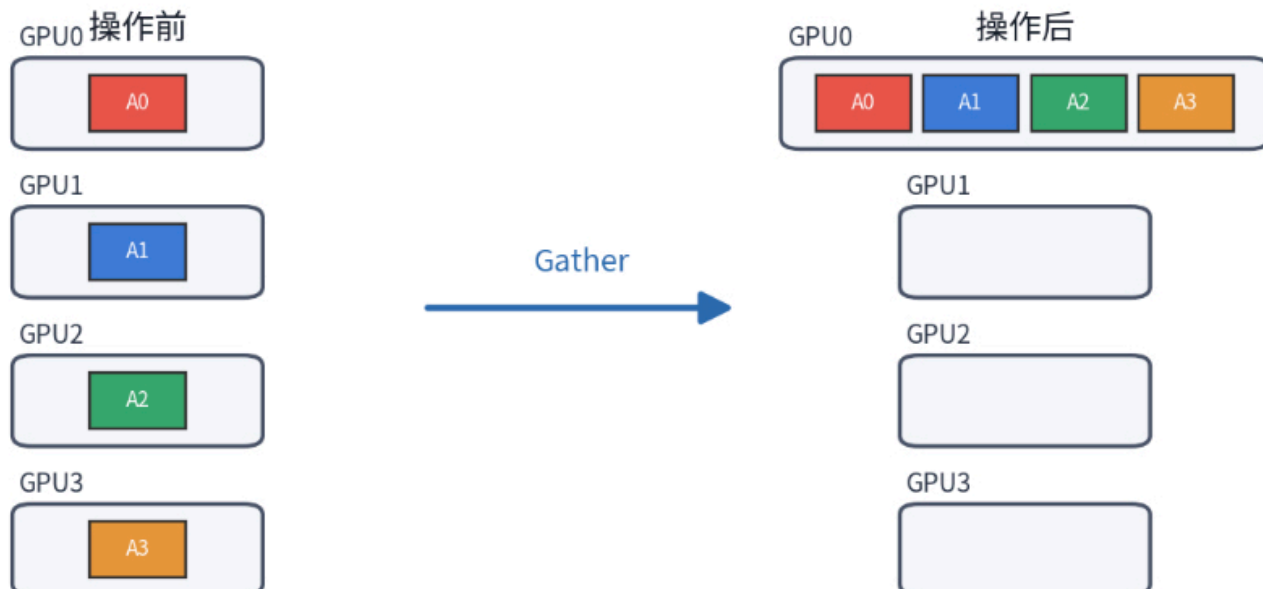
GPU0 上的数组被切分为 4 个分片， A_i 分发到 GPU i ；与 Broadcast 的区别在于各卡收到的是不同的分片。

Gather（收集，多对1）

把各 GPU 的分片汇聚拼接到根节点，只搬运不计算，是 Scatter 的逆操作。

Gather 收集

多对 1：把各 GPU 的数据汇聚到根节点（Scatter 的逆操作）



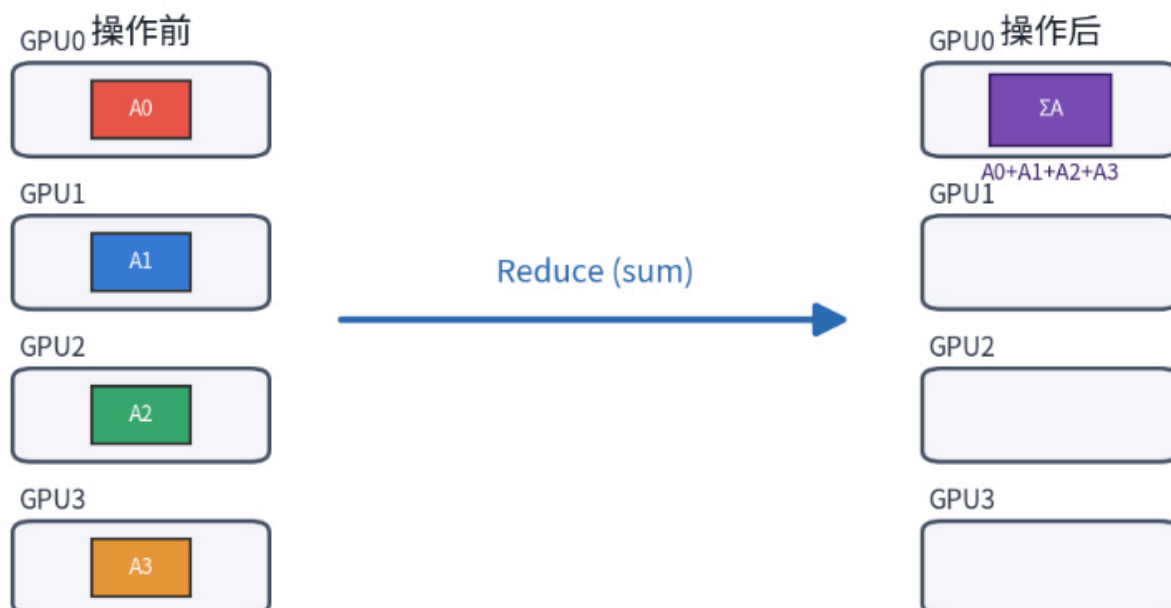
每张 GPU 各自的分片 A_i 被汇聚拼接到根 GPU0；只做搬运不做计算，是 Scatter 的逆过程。

Reduce（规约，多对1）

把各 GPU 的数据按元素做规约运算（默认求和，也可 max/min/prod），结果只落在根 GPU 上。

Reduce 规约

多对 1：把各 GPU 数据按位做规约（求和等）后汇总到根节点



各 GPU 的数据按元素做规约运算（默认求和，也可 max/min/prod），结果只落在根 GPU0 上。

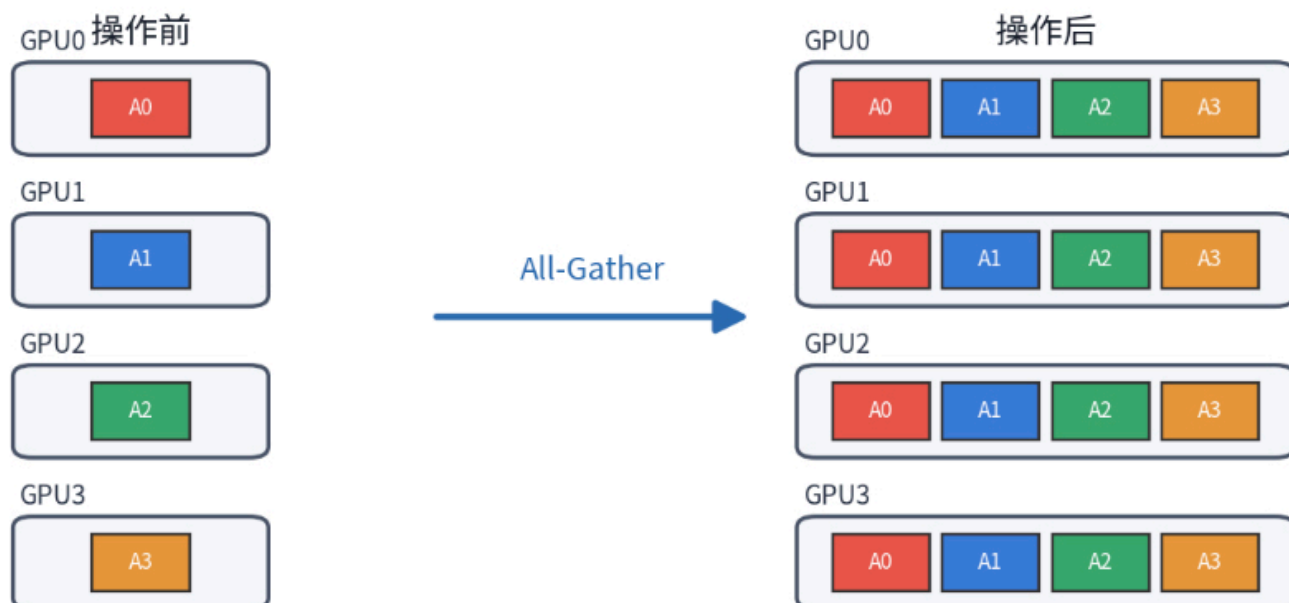
2. 多对多的组合原语

All-Gather（全收集，多对多）

每张卡贡献一个分片，操作后所有卡都持有完整拼接数据。等价于 Gather + Broadcast。常见于 ZeRO/FSDP 参数聚合、张量并行结果拼接。

All-Gather 全收集

多对多：每张 GPU 都收集到全部分片（Gather 后再 Broadcast）



每张 GPU 贡献一个分片 A_i ，操作后所有 GPU 都持有拼接好的完整数据 $[A_0, A_1, A_2, A_3]$ 。

All-Reduce（全规约，多对多）

各卡数据按元素规约（求和）后，结果同步回每一张卡。等价于 Reduce + Broadcast，是数据并行梯度同步最核心的原语，工程上常用 Ring-AllReduce 实现以均衡带宽。

All-Reduce 全规约

多对多：规约求和后把结果同步回每张 GPU（数据并行梯度同步核心）



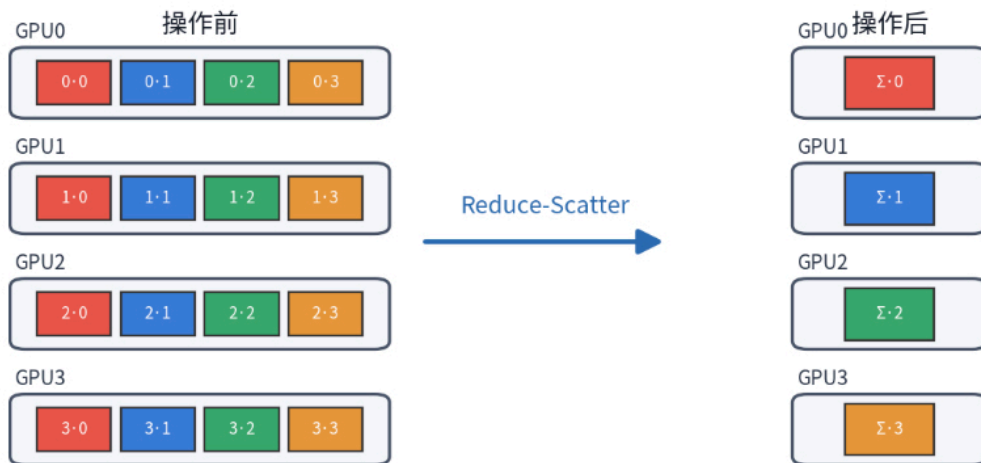
各 GPU 数据按元素规约（求和）后，结果 ΣA 同步到每一张 GPU；等价于 Reduce+Broadcast，是梯度同步最常用的原语。

Reduce-Scatter（规约散播，多对多）

先对所有卡的对应分段做规约，再把第 i 段结果散播给 GPU i （每卡只留一段）。等价于 Reduce + Scatter，是 **All-Reduce** 的前半程（Ring-AllReduce = ReduceScatter + AllGather）

Reduce-Scatter 规约散播

多对多：先按位规约，再把结果的第 i 段散播给 GPU i



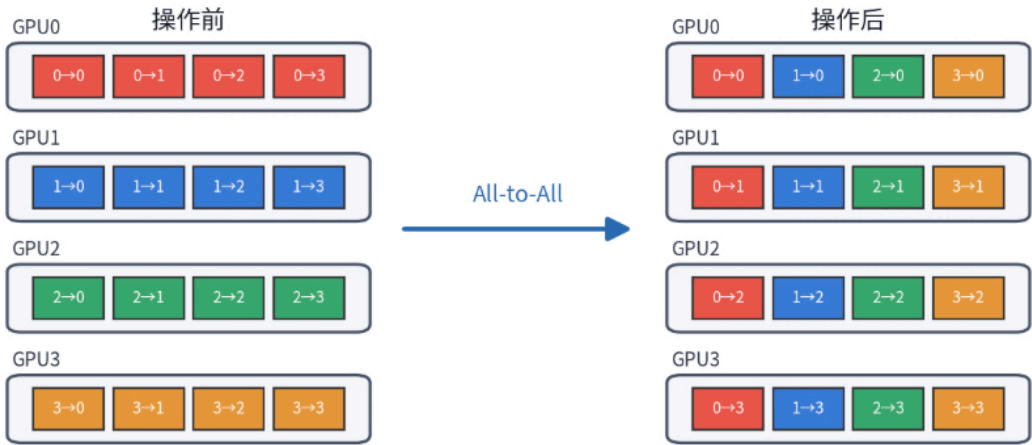
先对所有 GPU 的对应分段做规约（求和），再把第 i 段结果散播到 GPU i ；相当于 Reduce + Scatter，是 All-Reduce 的前半程。

All-to-All（全交换，多对多）

每张卡把自己的第 j 个分片发给 GPU j ，同时接收来自所有卡的分片，数据整体发生转置。是 **MoE 专家并行（EP）** 的关键通信，也用于序列并行/上下文并行的重排。

All-to-All 全交换

多对多：GPU i 的第 j 段发给 GPU j，等价于分块矩阵转置



每张 GPU 把自己的第 j 个分片发送给 GPU j，同时接收来自所有 GPU 的分片；数据整体发生转置，是 MoE 专家并行(EP) 的关键通信。

参考资料

1	并行策略	文章名	链接
2	综合/混合	Distributed Hybrid Parallelism for LLMs (arXiv 2602.09109)	https://arxiv.org/abs/2602.09109
3	综合/混合	Efficient Training of LLMs on Distributed Infrastructures Survey (arXiv 2407.20018)	https://arxiv.org/abs/2407.20018
4	综合/混合	Comprehensive Analysis of LLM Inference Parallelism	https://wilsonwu.me/en/blog/2024/08/comprehensive-analysis-of-llm-inference-parallelism-in-vllm/
5	综合/混合	BentoML LLM Inference Handbook: Hybrid Parallelism	https://bentoml.com/llm/inferer/optimization/data-tensor-pipeline/hybrid-parallelism
6	综合/混合	大模型并行技术理解 DP/TP/PP/EP/SP/CP (知乎)	https://zhuanlan.zhihu.com/p/4828609
7	综合/混合	一文讲清 AI大模型的并行训练方式 (知乎)	https://zhuanlan.zhihu.com/p/19393876
8	综合/混合	5D Parallelism 详解DP/TP/PP/CP/EP (知乎)	https://zhuanlan.zhihu.com/p/6190513
9	综合/混合	从 1 到 16 卡：SGLang 推理中的 Chunked Prefill 与四种并行技术全解析	从 1 到 16 卡：SGLang 推理中的 Chunked Prefill 与四种并行技术全解析

10	DP	大模型推理部署并行策略详解（CSDN）	https://blog.csdn.net/2601_953details/159802000
11	TP/CP/EP	Scaling LLM Inference (Meta Engineering)	https://engineering.fb.com/202research/scaling-llm-inference
12	TP/PP/DP/EP/CP	Parallelism in TensorRT-LLM (NVIDIA)	https://nvidia.github.io/TensorFLLM/features/parallel-strategy
13	PP	SGLang 超长上下文 Chunked Pipeline（知乎）	https://zhuanlan.zhihu.com/p/2620930
14	EP	vLLM MoE Playbook: TP/DP/PP/EP (AMD ROCm)	https://rocm.blogs.amd.com/scoptimization/vllm-moe-guide/R
15	CP	DeepSeek-V3.2 128K + 百度百舸 CP 方案	https://developer.baidu.com/ar?id=5417181
16	CP	vLLM 并行策略之CP (Context Parallel)（知乎）	https://zhuanlan.zhihu.com/p/20378281